

Tower defense game

Generated by Doxygen 1.9.1

1 Source content	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Class Documentation	9
5.1 BasicEnemy Class Reference	9
5.1.1 Detailed Description	10
5.1.2 Constructor & Destructor Documentation	10
5.1.2.1 BasicEnemy()	10
5.2 basicProjectile Class Reference	10
5.2.1 Detailed Description	11
5.2.2 Constructor & Destructor Documentation	11
5.2.2.1 basicProjectile()	11
5.3 basicTower Class Reference	12
5.3.1 Detailed Description	13
5.3.2 Constructor & Destructor Documentation	13
5.3.2.1 basicTower()	13
5.3.2.2 ~basicTower()	13
5.3.3 Member Function Documentation	13
5.3.3.1 fire()	14
5.3.3.2 getProjectiles()	14
5.3.4 Member Data Documentation	14
5.3.4.1 projectiles_	14
5.4 BombProjectile Class Reference	15
5.4.1 Detailed Description	15
5.4.2 Constructor & Destructor Documentation	16
5.4.2.1 BombProjectile()	16
5.5 BombTower Class Reference	16
5.5.1 Detailed Description	17
5.5.2 Constructor & Destructor Documentation	17
5.5.2.1 BombTower()	17
5.5.2.2 ~BombTower()	18
5.5.3 Member Function Documentation	18
5.5.3.1 fire()	18
5.5.3.2 getProjectiles()	18
5.5.4 Member Data Documentation	18
5.5.4.1 projectiles_	18

5.6 BossEnemy Class Reference	19
5.6.1 Detailed Description	19
5.6.2 Constructor & Destructor Documentation	20
5.6.2.1 BossEnemy()	20
5.7 Button Class Reference	20
5.7.1 Detailed Description	21
5.7.2 Constructor & Destructor Documentation	21
5.7.2.1 Button()	21
5.7.3 Member Function Documentation	22
5.7.3.1 getSprite()	22
5.7.3.2 getText()	22
5.7.3.3 getType()	22
5.7.3.4 isClicked()	22
5.7.3.5 isFinnish()	23
5.7.3.6 isStart()	23
5.7.3.7 setClicked()	23
5.7.3.8 setFinnish()	23
5.7.3.9 setNotClicked()	23
5.7.3.10 setNotFinnish()	24
5.7.3.11 setNotStart()	24
5.7.3.12 setSprite()	24
5.7.3.13 setStart()	24
5.7.4 Member Data Documentation	24
5.7.4.1 clicked_	24
5.7.4.2 font_	25
5.7.4.3 is_clicked_	25
5.7.4.4 is_Finnish	25
5.7.4.5 is_Start	25
5.7.4.6 notClicked_	25
5.7.4.7 sprite_	25
5.7.4.8 text_	25
5.7.4.9 texture_	25
5.7.4.10 type_	26
5.8 Enemy Class Reference	26
5.8.1 Detailed Description	28
5.8.2 Constructor & Destructor Documentation	28
5.8.2.1 Enemy()	28
5.8.2.2 ~Enemy()	28
5.8.3 Member Function Documentation	28
5.8.3.1 atCheckpoint()	28
5.8.3.2 checkFreeze()	29
5.8.3.3 checkPoisonDamage()	29

5.8.3.4 Freeze()	29
5.8.3.5 getHitbox()	29
5.8.3.6 getHitpoints()	29
5.8.3.7 getPoisoned()	30
5.8.3.8 getSpeed()	30
5.8.3.9 getSprite()	30
5.8.3.10 getSteps()	30
5.8.3.11 getType()	31
5.8.3.12 isalive()	31
5.8.3.13 isFrozen()	31
5.8.3.14 kill()	31
5.8.3.15 move()	32
5.8.3.16 setCheckpoint()	32
5.8.3.17 setHitpoints()	32
5.8.3.18 shouldPay()	32
5.8.3.19 takeDamage()	33
5.8.3.20 unFreeze()	34
5.8.4 Member Data Documentation	34
5.8.4.1 alive_	34
5.8.4.2 death_sound_	34
5.8.4.3 death_sound_buffer_	34
5.8.4.4 delta_x	34
5.8.4.5 delta_y	34
5.8.4.6 freeze_timer_	35
5.8.4.7 frozen_	35
5.8.4.8 has_payed_	35
5.8.4.9 hitbox_	35
5.8.4.10 hitpoints_	35
5.8.4.11 next_point_	35
5.8.4.12 poison_timer_	35
5.8.4.13 poisoned_	35
5.8.4.14 speed_	36
5.8.4.15 sprite_	36
5.8.4.16 steps_taken_	36
5.8.4.17 texture_	36
5.8.4.18 type_	36
5.8.4.19 x_goal_	36
5.8.4.20 y_goal_	36
5.9 Game Class Reference	37
5.9.1 Detailed Description	38
5.9.2 Constructor & Destructor Documentation	38
5.9.2.1 Game()	38

5.9.2.2 ~Game()	38
5.9.3 Member Function Documentation	38
5.9.3.1 checkButtons()	38
5.9.3.2 checkPositions()	39
5.9.3.3 checkTowers()	39
5.9.3.4 intersects()	39
5.9.3.5 moveTower()	40
5.9.3.6 pollEv()	40
5.9.3.7 render()	40
5.9.3.8 running()	40
5.9.3.9 showErrorMessage()	40
5.9.3.10 showInfo()	41
5.9.3.11 update()	41
5.9.3.12 validPlacement()	41
5.9.4 Member Data Documentation	41
5.9.4.1 button_background_	42
5.9.4.2 buttons_	42
5.9.4.3 clock	42
5.9.4.4 enemies_	42
5.9.4.5 event_	42
5.9.4.6 map_	42
5.9.4.7 nodes_	42
5.9.4.8 paused_	42
5.9.4.9 placing_new_tower	43
5.9.4.10 player_	43
5.9.4.11 state_	43
5.9.4.12 tiles_	43
5.9.4.13 towers_	43
5.9.4.14 wave_number_	43
5.9.4.15 window_	43
5.10 GameCell Class Reference	44
5.10.1 Constructor & Destructor Documentation	44
5.10.1.1 GameCell()	44
5.10.1.2 ~GameCell()	45
5.10.2 Member Function Documentation	45
5.10.2.1 getSprite()	45
5.10.2.2 setUpSprite()	45
5.10.3 Member Data Documentation	45
5.10.3.1 isExit_	45
5.10.3.2 isPassable_	46
5.10.3.3 isStart_	46
5.10.3.4 pos_	46

5.10.3.5 sprite_	46
5.10.3.6 texture_	46
5.11 GameOver Class Reference	46
5.11.1 Detailed Description	47
5.11.2 Constructor & Destructor Documentation	47
5.11.2.1 GameOver()	47
5.11.3 Member Function Documentation	47
5.11.3.1 checkButtons()	47
5.11.3.2 pollEv()	48
5.11.3.3 render()	48
5.11.3.4 running()	48
5.11.3.5 update()	48
5.11.4 Member Data Documentation	48
5.11.4.1 buttons_	49
5.11.4.2 event_	49
5.11.4.3 font_	49
5.11.4.4 state_	49
5.11.4.5 text_	49
5.11.4.6 window_	49
5.12 IceProjectile Class Reference	50
5.12.1 Detailed Description	50
5.12.2 Constructor & Destructor Documentation	51
5.12.2.1 IceProjectile()	51
5.13 IceTower Class Reference	51
5.13.1 Detailed Description	52
5.13.2 Constructor & Destructor Documentation	52
5.13.2.1 IceTower()	52
5.13.2.2 ~IceTower()	53
5.13.3 Member Function Documentation	53
5.13.3.1 fire()	53
5.13.3.2 getProjectiles()	53
5.13.4 Member Data Documentation	53
5.13.4.1 projectiles_	53
5.14 mainMenu Class Reference	54
5.14.1 Constructor & Destructor Documentation	54
5.14.1.1 mainMenu()	54
5.14.1.2 ~mainMenu()	55
5.14.2 Member Function Documentation	55
5.14.2.1 getNextWindow()	55
5.14.2.2 isRunning()	55
5.14.2.3 pollEv()	55
5.14.2.4 Render()	55

5.14.2.5 Update()	55
5.14.3 Member Data Documentation	56
5.14.3.1 buttons_	56
5.14.3.2 event_	56
5.14.3.3 exitButton_	56
5.14.3.4 exitButtonPressed_	56
5.14.3.5 exitButtonSelected_	56
5.14.3.6 font_	56
5.14.3.7 gameTitle_	56
5.14.3.8 mapButton_	57
5.14.3.9 mapButtonPressed_	57
5.14.3.10 mapButtonSelected_	57
5.14.3.11 next_window_	57
5.14.3.12 playButton_	57
5.14.3.13 playButtonPressed_	57
5.14.3.14 playButtonSelected_	57
5.14.3.15 window_	58
5.15 Map Class Reference	58
5.15.1 Constructor & Destructor Documentation	58
5.15.1.1 Map()	58
5.15.1.2 ~Map()	59
5.15.2 Member Function Documentation	59
5.15.2.1 getNodes()	59
5.15.2.2 getTiles()	59
5.15.2.3 setUpCells()	59
5.15.3 Member Data Documentation	59
5.15.3.1 file_path_	60
5.15.3.2 nodes_	60
5.15.3.3 tile_matrix_	60
5.16 mapCreator Class Reference	60
5.16.1 Constructor & Destructor Documentation	61
5.16.1.1 mapCreator()	61
5.16.1.2 ~mapCreator()	61
5.16.2 Member Function Documentation	61
5.16.2.1 checkButtons()	61
5.16.2.2 isRunning()	61
5.16.2.3 pollEv()	62
5.16.2.4 Render()	62
5.16.2.5 showInfo()	62
5.16.2.6 Update()	62
5.16.3 Member Data Documentation	62
5.16.3.1 buttons_	62

5.16.3.2 event_	62
5.16.3.3 mainMenuButton_	63
5.16.3.4 saveButton_	63
5.16.3.5 setFinnishButton_	63
5.16.3.6 setStartButton_	63
5.16.3.7 window_	63
5.17 Node Class Reference	63
5.17.1 Constructor & Destructor Documentation	64
5.17.1.1 Node()	64
5.17.2 Member Function Documentation	64
5.17.2.1 get_cord()	64
5.17.2.2 get_next()	64
5.17.2.3 set_cord()	64
5.17.2.4 set_next()	65
5.17.3 Member Data Documentation	65
5.17.3.1 coordinate_	65
5.17.3.2 next_	65
5.18 Player Class Reference	65
5.18.1 Detailed Description	66
5.18.2 Constructor & Destructor Documentation	66
5.18.2.1 Player()	66
5.18.3 Member Function Documentation	67
5.18.3.1 getHealth()	67
5.18.3.2 getHealthText()	67
5.18.3.3 getMoney()	67
5.18.3.4 getMoneyText()	67
5.18.3.5 giveMoney()	67
5.18.3.6 heal()	68
5.18.3.7 takeDamage()	68
5.18.3.8 takeMoney()	68
5.18.4 Member Data Documentation	69
5.18.4.1 font_	69
5.18.4.2 health_	69
5.18.4.3 hp_text_	69
5.18.4.4 money_	69
5.18.4.5 money_text_	69
5.19 PoisonProjectile Class Reference	70
5.19.1 Detailed Description	70
5.19.2 Constructor & Destructor Documentation	71
5.19.2.1 PoisonProjectile()	71
5.20 PoisonTower Class Reference	71
5.20.1 Detailed Description	72

5.20.2 Constructor & Destructor Documentation	72
5.20.2.1 PoisonTower()	72
5.20.2.2 ~PoisonTower()	73
5.20.3 Member Function Documentation	73
5.20.3.1 fire()	73
5.20.3.2 getProjectiles()	73
5.20.4 Member Data Documentation	73
5.20.4.1 projectiles_	73
5.21 Projectile Class Reference	74
5.21.1 Detailed Description	75
5.21.2 Constructor & Destructor Documentation	75
5.21.2.1 Projectile()	75
5.21.2.2 ~Projectile()	75
5.21.3 Member Function Documentation	75
5.21.3.1 getDamage()	75
5.21.3.2 getEnemy()	76
5.21.3.3 getHitbox()	76
5.21.3.4 getSprite()	76
5.21.3.5 move()	76
5.21.4 Member Data Documentation	77
5.21.4.1 damage_	77
5.21.4.2 enemy_	77
5.21.4.3 hitbox_	77
5.21.4.4 speed_	77
5.21.4.5 sprite_	77
5.21.4.6 texture_	77
5.21.4.7 type_	77
5.22 StealthEnemy Class Reference	78
5.22.1 Detailed Description	78
5.22.2 Constructor & Destructor Documentation	79
5.22.2.1 StealthEnemy()	79
5.23 Tower Class Reference	79
5.23.1 Detailed Description	81
5.23.2 Constructor & Destructor Documentation	81
5.23.2.1 Tower()	81
5.23.2.2 ~Tower()	81
5.23.3 Member Function Documentation	81
5.23.3.1 fire()	82
5.23.3.2 getAttackSpeed()	82
5.23.3.3 getLastFireTime()	82
5.23.3.4 getLevel()	82
5.23.3.5 getProjectiles()	83

5.23.3.6	getRange()	83
5.23.3.7	getRangeCircle()	83
5.23.3.8	getSprite()	83
5.23.3.9	getTarget()	84
5.23.3.10	getText()	84
5.23.3.11	getType()	84
5.23.3.12	hasTarget()	84
5.23.3.13	isSelected()	85
5.23.3.14	removeTarget()	85
5.23.3.15	resetSelected()	85
5.23.3.16	setLastFireTime()	85
5.23.3.17	setSelected()	85
5.23.3.18	setTarget()	86
5.23.3.19	upgrade()	86
5.23.4	Member Data Documentation	86
5.23.4.1	attack_speed_	86
5.23.4.2	font_	86
5.23.4.3	has_target_	86
5.23.4.4	last_fire_time_	86
5.23.4.5	level_	87
5.23.4.6	price_	87
5.23.4.7	range_	87
5.23.4.8	range_circle_	87
5.23.4.9	selected_	87
5.23.4.10	sprite_	87
5.23.4.11	target_	87
5.23.4.12	text_	87
5.23.4.13	texture_	88
5.23.4.14	type_	88
5.24	Wave Class Reference	88
5.24.1	Detailed Description	88
5.24.2	Constructor & Destructor Documentation	88
5.24.2.1	Wave()	88
5.24.3	Member Function Documentation	89
5.24.3.1	getEnemies()	89
5.24.4	Member Data Documentation	89
5.24.4.1	enemies_	89
6	File Documentation	91
6.1	src/Basic_enemy.cpp File Reference	91
6.2	src/Basic_enemy.hpp File Reference	91
6.3	src/Basic_projectile.cpp File Reference	92

6.4 src/Basic_projectile.hpp File Reference	93
6.5 src/Basic_tower.cpp File Reference	94
6.6 src/Basic_tower.hpp File Reference	94
6.7 src/Bomb_projectile.cpp File Reference	95
6.8 src/Bomb_projectile.hpp File Reference	96
6.9 src/Bomb_tower.cpp File Reference	97
6.10 src/Bomb_tower.hpp File Reference	97
6.11 src/Boss_enemy.cpp File Reference	98
6.12 src/Boss_enemy.hpp File Reference	99
6.13 src/Button.cpp File Reference	100
6.14 src/Button.hpp File Reference	101
6.14.1 Detailed Description	102
6.15 src/Enemy.cpp File Reference	102
6.16 src/Enemy.hpp File Reference	103
6.16.1 Detailed Description	103
6.17 src/Game.cpp File Reference	104
6.18 src/Game.hpp File Reference	104
6.19 src/Game_over.cpp File Reference	105
6.20 src/Game_over.hpp File Reference	106
6.20.1 Detailed Description	107
6.21 src/gameCell.cpp File Reference	108
6.22 src/gameCell.hpp File Reference	108
6.22.1 Detailed Description	109
6.23 src/Ice_projectile.cpp File Reference	110
6.24 src/Ice_projectile.hpp File Reference	110
6.25 src/Ice_tower.cpp File Reference	111
6.26 src/Ice_tower.hpp File Reference	112
6.27 src/main.cpp File Reference	113
6.27.1 Function Documentation	113
6.27.1.1 main()	113
6.28 src/mainMenu.cpp File Reference	114
6.29 src/mainMenu.hpp File Reference	114
6.29.1 Detailed Description	115
6.30 src/Map.cpp File Reference	115
6.31 src/Map.hpp File Reference	116
6.31.1 Detailed Description	117
6.32 src/mapCreator.cpp File Reference	117
6.33 src/mapCreator.hpp File Reference	117
6.33.1 Detailed Description	118
6.34 src/Node.cpp File Reference	119
6.35 src/Node.hpp File Reference	119
6.36 src/pathfinding.cpp File Reference	120

6.36.1 Function Documentation	120
6.36.1.1 isValid()	120
6.36.1.2 pathSearch()	121
6.37 src/pathfinding.hpp File Reference	121
6.37.1 Function Documentation	122
6.37.1.1 isValid()	122
6.37.1.2 pathSearch()	123
6.38 src/Player.cpp File Reference	123
6.39 src/Player.hpp File Reference	123
6.39.1 Detailed Description	124
6.40 src/Poison_projectile.cpp File Reference	125
6.41 src/Poison_projectile.hpp File Reference	125
6.42 src/Poison_tower.cpp File Reference	126
6.43 src/Poison_tower.hpp File Reference	127
6.44 src/Projectile.cpp File Reference	128
6.45 src/Projectile.hpp File Reference	129
6.46 src/readme.md File Reference	129
6.47 src/Stealth_enemy.cpp File Reference	129
6.48 src/Stealth_enemy.hpp File Reference	130
6.49 src/Tower.cpp File Reference	131
6.50 src/Tower.hpp File Reference	132
6.51 src/Wave.cpp File Reference	133
6.52 src/Wave.hpp File Reference	133
6.52.1 Detailed Description	134
Index	137

Chapter 1

Source content

This folder should contain only `hpp/cpp` files of your implementation. You can also place `hpp` files in a separate directory `include`.

You can create a summary of files here. It might be useful to describe file relations, and brief summary of their content.

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Button	20
Enemy	26
BasicEnemy	9
BossEnemy	19
StealthEnemy	78
Game	37
GameCell	44
GameOver	46
mainMenu	54
Map	58
mapCreator	60
Node	63
Player	65
Projectile	74
BombProjectile	15
IceProjectile	50
PoisonProjectile	70
basicProjectile	10
Tower	79
BombTower	16
IceTower	51
PoisonTower	71
basicTower	12
Wave	88

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BasicEnemy	Child class for basic enemies	9
basicProjectile	Basic projectile for basic tower	10
basicTower	Basic tower class	12
BombProjectile	Bomb projectile for bomb tower	15
BombTower	Bomb tower class	16
BossEnemy	Child class for boss enemies	19
Button	Class for implementing buttons using SFML	20
Enemy	Enemy parent class	26
Game		37
GameCell		44
GameOver	Handles state after game finishes execution	46
IceProjectile	Ice projectile for ice tower	50
IceTower	Ice tower class	51
mainMenu		54
Map		58
mapCreator		60
Node		63
Player	Class for handling gamestate from player perspective	65
PoisonProjectile	Poison projectile for poison tower	70
PoisonTower	Poison tower class	71
Projectile	Parent class for projectiles	74

StealthEnemy

Child class for stealth enemies 78

Tower

Parent class for different towers 79

Wave

Class that generates individual waves 88

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

src/Basic_enemy.cpp	91
src/Basic_enemy.hpp	91
src/Basic_projectile.cpp	92
src/Basic_projectile.hpp	93
src/Basic_tower.cpp	94
src/Basic_tower.hpp	94
src/Bomb_projectile.cpp	95
src/Bomb_projectile.hpp	96
src/Bomb_tower.cpp	97
src/Bomb_tower.hpp	97
src/Boss_enemy.cpp	98
src/Boss_enemy.hpp	99
src/Button.cpp	100
src/Button.hpp	101
src/Enemy.cpp	102
src/Enemy.hpp	
Parent class for all enemies	103
src/Game.cpp	104
src/Game.hpp	104
src/Game_over.cpp	105
src/Game_over.hpp	
Handles state after game finishes execution	106
src/gameCell.cpp	108
src/gameCell.hpp	
Splitting the map into cells that are easier to handle	108
src/Ice_projectile.cpp	110
src/Ice_projectile.hpp	110
src/Ice_tower.cpp	111
src/Ice_tower.hpp	112
src/main.cpp	113
src/mainMenu.cpp	114
src/mainMenu.hpp	
Main Menu Implementation	114
src/Map.cpp	115
src/Map.hpp	
Map Class Implementation	116

src/ mapCreator.cpp	117
src/ mapCreator.hpp	
Map Creator Implementation	117
src/ Node.cpp	119
src/ Node.hpp	119
src/ pathfinding.cpp	120
src/ pathfinding.hpp	121
src/ Player.cpp	123
src/ Player.hpp	
Class for handling status of player	123
src/ Poison_projectile.cpp	125
src/ Poison_projectile.hpp	125
src/ Poison_tower.cpp	126
src/ Poison_tower.hpp	127
src/ Projectile.cpp	128
src/ Projectile.hpp	129
src/ Stealth_enemy.cpp	129
src/ Stealth_enemy.hpp	130
src/ Tower.cpp	131
src/ Tower.hpp	132
src/ Wave.cpp	133
src/ Wave.hpp	133

Chapter 5

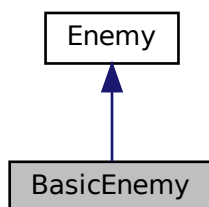
Class Documentation

5.1 BasicEnemy Class Reference

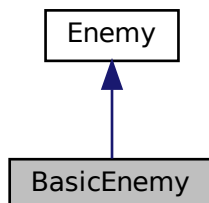
Child class for basic enemies.

```
#include <Basic_enemy.hpp>
```

Inheritance diagram for BasicEnemy:



Collaboration diagram for BasicEnemy:



Public Member Functions

- [BasicEnemy](#) (std::shared_ptr< [Node](#) > n)
Construct a new Basic [Enemy](#) object.

Additional Inherited Members

5.1.1 Detailed Description

Child class for basic enemies.

5.1.2 Constructor & Destructor Documentation

5.1.2.1 BasicEnemy()

```
BasicEnemy::BasicEnemy (  
    std::shared_ptr< Node > n )
```

Construct a new Basic [Enemy](#) object.

Parameters

<i>n</i>	starting node
----------	---------------

The documentation for this class was generated from the following files:

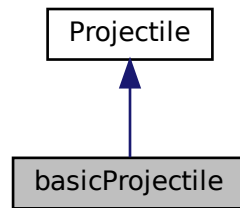
- src/[Basic_enemy.hpp](#)
- src/[Basic_enemy.cpp](#)

5.2 basicProjectile Class Reference

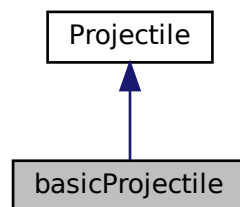
Basic projectile for basic tower.

```
#include <Basic_projectile.hpp>
```


Inheritance diagram for basicProjectile:



Collaboration diagram for basicProjectile:



Public Member Functions

- `basicProjectile` (`std::shared_ptr< Enemy > enemy`, `sf::Vector2f origin`)
Construct a new basic `Projectile` object.

Additional Inherited Members

5.2.1 Detailed Description

Basic projectile for basic tower.

5.2.2 Constructor & Destructor Documentation

5.2.2.1 basicProjectile()

```

basicProjectile::basicProjectile (
    std::shared_ptr< Enemy > enemy,
    sf::Vector2f origin )

```

Construct a new basic `Projectile` object.

Parameters

<i>enemy</i>	target
<i>origin</i>	position of tower

The documentation for this class was generated from the following files:

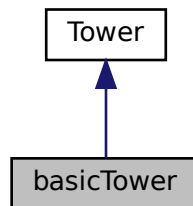
- [src/Basic_projectile.hpp](#)
- [src/Basic_projectile.cpp](#)

5.3 basicTower Class Reference

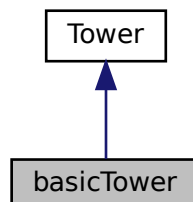
Basic tower class.

```
#include <Basic_tower.hpp>
```

Inheritance diagram for basicTower:



Collaboration diagram for basicTower:



Public Member Functions

- [basicTower](#) ()
Construct a new basic [Tower](#) object.
- [~basicTower](#) ()
Destroy the basic [Tower](#) object.
- void [fire](#) ()
Fire at enemy.
- std::vector< std::shared_ptr< [Projectile](#) > > & [getProjectiles](#) ()
Get the [Projectiles](#) object.

Private Attributes

- std::vector< std::shared_ptr< [Projectile](#) > > [projectiles_](#)

Additional Inherited Members

5.3.1 Detailed Description

Basic tower class.

5.3.2 Constructor & Destructor Documentation

5.3.2.1 basicTower()

```
basicTower::basicTower ( )
```

Construct a new basic [Tower](#) object.

5.3.2.2 ~basicTower()

```
basicTower::~~basicTower ( )
```

Destroy the basic [Tower](#) object.

5.3.3 Member Function Documentation

5.3.3.1 fire()

```
void basicTower::fire ( ) [virtual]
```

Fire at enemy.

Implements [Tower](#).

5.3.3.2 getProjectiles()

```
std::vector< std::shared_ptr< Projectile > > & basicTower::getProjectiles ( ) [virtual]
```

Get the Projectiles object.

Returns

```
std::vector<std::shared_ptr<Projectile>>&
```

Implements [Tower](#).

5.3.4 Member Data Documentation

5.3.4.1 projectiles_

```
std::vector<std::shared_ptr<Projectile> > basicTower::projectiles_ [private]
```

The documentation for this class was generated from the following files:

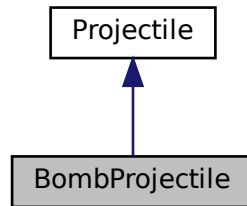
- [src/Basic_tower.hpp](#)
- [src/Basic_tower.cpp](#)

5.4 BombProjectile Class Reference

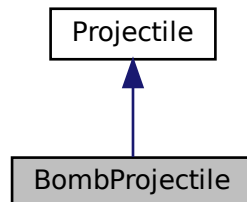
Bomb projectile for bomb tower.

```
#include <Bomb_projectile.hpp>
```

Inheritance diagram for BombProjectile:



Collaboration diagram for BombProjectile:



Public Member Functions

- [BombProjectile](#) (std::shared_ptr< [Enemy](#) > enemy, sf::Vector2f origin)
Construct a new Bomb [Projectile](#) object.

Additional Inherited Members

5.4.1 Detailed Description

Bomb projectile for bomb tower.

5.4.2 Constructor & Destructor Documentation

5.4.2.1 BombProjectile()

```
BombProjectile::BombProjectile (
    std::shared_ptr< Enemy > enemy,
    sf::Vector2f origin )
```

Construct a new Bomb [Projectile](#) object.

Parameters

<i>enemy</i>	target
<i>origin</i>	position of tower

The documentation for this class was generated from the following files:

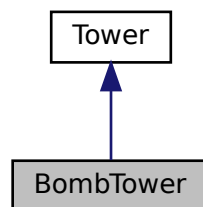
- [src/Bomb_projectile.hpp](#)
- [src/Bomb_projectile.cpp](#)

5.5 BombTower Class Reference

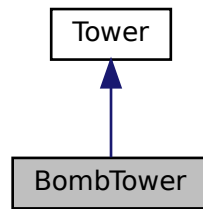
Bomb tower class.

```
#include <Bomb_tower.hpp>
```

Inheritance diagram for BombTower:



Collaboration diagram for BombTower:



Public Member Functions

- [BombTower](#) ()
Construct a new Bomb [Tower](#) object.
- [~BombTower](#) ()
Destroy the Bomb [Tower](#) object.
- void [fire](#) ()
Fire at enemy git.
- std::vector< std::shared_ptr< [Projectile](#) > > & [getProjectiles](#) ()
Get the Projectiles object.

Private Attributes

- std::vector< std::shared_ptr< [Projectile](#) > > [projectiles_](#)

Additional Inherited Members

5.5.1 Detailed Description

Bomb tower class.

5.5.2 Constructor & Destructor Documentation

5.5.2.1 BombTower()

```
BombTower::BombTower ( )
```

Construct a new Bomb [Tower](#) object.

5.5.2.2 ~BombTower()

```
BombTower::~~BombTower ( )
```

Destroy the Bomb [Tower](#) object.

5.5.3 Member Function Documentation

5.5.3.1 fire()

```
void BombTower::fire ( ) [virtual]
```

Fire at enemy git.

Implements [Tower](#).

5.5.3.2 getProjectiles()

```
std::vector< std::shared_ptr< Projectile > > & BombTower::getProjectiles ( ) [virtual]
```

Get the Projectiles object.

Returns

```
std::vector<std::shared_ptr<Projectile>>&
```

Implements [Tower](#).

5.5.4 Member Data Documentation

5.5.4.1 projectiles_

```
std::vector<std::shared_ptr<Projectile> > BombTower::projectiles_ [private]
```

The documentation for this class was generated from the following files:

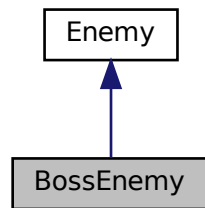
- [src/Bomb_tower.hpp](#)
- [src/Bomb_tower.cpp](#)

5.6 BossEnemy Class Reference

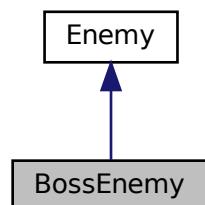
Child class for boss enemies.

```
#include <Boss_enemy.hpp>
```

Inheritance diagram for BossEnemy:



Collaboration diagram for BossEnemy:



Public Member Functions

- `BossEnemy` (`std::shared_ptr< Node > n`)
Construct a new Boss `Enemy` object.

Additional Inherited Members

5.6.1 Detailed Description

Child class for boss enemies.

5.6.2 Constructor & Destructor Documentation

5.6.2.1 BossEnemy()

```

BossEnemy::BossEnemy (
    std::shared_ptr< Node > n )

```

Construct a new Boss [Enemy](#) object.

Parameters

<i>n</i>	starting node
----------	---------------

The documentation for this class was generated from the following files:

- src/[Boss_enemy.hpp](#)
- src/[Boss_enemy.cpp](#)

5.7 Button Class Reference

Class for implementing buttons using SFML.

```
#include <Button.hpp>
```

Public Member Functions

- [Button](#) (std::string texturePathN, std::string texturePathC, std::string text, int x_cord, int y_cord, int type)
Construct a new [Button](#) object.
- sf::Sprite & [getSprite](#) ()
Get the Sprite object.
- sf::Text & [getText](#) ()
Get the Text object.
- void [setClicked](#) ()
Set the button to clicked state.
- void [setNotClicked](#) ()
Set the button to not clicked state.
- bool [isClicked](#) ()
Check if the button has been clicked.
- int [getType](#) ()
Get the type of button.
- void [setStart](#) ()
Set the button as a start button.
- void [setFinnish](#) ()
Set the buttons as a finish button.
- void [setNotStart](#) ()

- *Set the button to not be a start button.*
- void `setNotFinnish` ()
- *Set the button to not be a finish button.*
- bool `isStart` ()
- *Check if button is a start button.*
- bool `isFinnish` ()
- *Check if button is a finish button.*
- void `setSprite` (std::string path)
- *Set the sprite for the button.*

Private Attributes

- int `type_`
- bool `is_clicked_` = false
- bool `is_Start` = false
- bool `is_Finnish` = false
- sf::Text `text_`
- sf::Font `font_`
- sf::Texture `texture_`
- sf::Sprite `sprite_`
- std::string `clicked_`
- std::string `notClicked_`

5.7.1 Detailed Description

Class for implementing buttons using SFML.

5.7.2 Constructor & Destructor Documentation

5.7.2.1 Button()

```
Button::Button (
    std::string texturePathN,
    std::string texturePathC,
    std::string text,
    int x_cord,
    int y_cord,
    int type )
```

Construct a new `Button` object.

Parameters

<i>texturePathN</i>	Texture path for not clicked button
<i>texturePathC</i>	Texture path for clicked button
<i>text</i>	Text on button
<i>x_cord</i>	X-coordinate of the button
<i>y_cord</i>	Y-coordinate of the button
<i>type</i>	Type of button

5.7.3 Member Function Documentation

5.7.3.1 getSprite()

```
sf::Sprite & Button::getSprite ( )
```

Get the Sprite object.

Returns

sf::Sprite&

5.7.3.2 getText()

```
sf::Text & Button::getText ( )
```

Get the Text object.

Returns

sf::Text&

5.7.3.3 getType()

```
int Button::getType ( )
```

Get the type of button.

Returns

int

5.7.3.4 isClicked()

```
bool Button::isClicked ( )
```

Check if the button has been clicked.

Returns

true

false

5.7.3.5 isFinnish()

```
bool Button::isFinnish ( )
```

Check if button is a finish button.

Returns

true
false

5.7.3.6 isStart()

```
bool Button::isStart ( )
```

Check if button is a start button.

Returns

true
false

5.7.3.7 setClicked()

```
void Button::setClicked ( )
```

Set the button to clicked state.

5.7.3.8 setFinnish()

```
void Button::setFinnish ( )
```

Set the buttons as a finish button.

5.7.3.9 setNotClicked()

```
void Button::setNotClicked ( )
```

Set the button to not clicked state.

5.7.3.10 setNotFinnish()

```
void Button::setNotFinnish ( )
```

Set the button to not be a finish button.

5.7.3.11 setNotStart()

```
void Button::setNotStart ( )
```

Set the button to not be a start button.

5.7.3.12 setSprite()

```
void Button::setSprite (
    std::string path )
```

Set the sprite for the button.

Parameters

<i>path</i>	path to the texture for the sprite
-------------	------------------------------------

5.7.3.13 setStart()

```
void Button::setStart ( )
```

Set the button as a start button.

5.7.4 Member Data Documentation

5.7.4.1 clicked_

```
std::string Button::clicked_ [private]
```

5.7.4.2 font_

```
sf::Font Button::font_ [private]
```

5.7.4.3 is_clicked_

```
bool Button::is_clicked_ = false [private]
```

5.7.4.4 is_Finnish

```
bool Button::is_Finnish = false [private]
```

5.7.4.5 is_Start

```
bool Button::is_Start = false [private]
```

5.7.4.6 notClicked_

```
std::string Button::notClicked_ [private]
```

5.7.4.7 sprite_

```
sf::Sprite Button::sprite_ [private]
```

5.7.4.8 text_

```
sf::Text Button::text_ [private]
```

5.7.4.9 texture_

```
sf::Texture Button::texture_ [private]
```

5.7.4.10 type_

```
int Button::type_ [private]
```

The documentation for this class was generated from the following files:

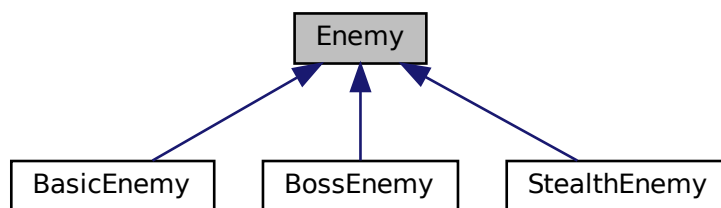
- [src/Button.hpp](#)
- [src/Button.cpp](#)

5.8 Enemy Class Reference

[Enemy](#) parent class.

```
#include <Enemy.hpp>
```

Inheritance diagram for Enemy:



Public Member Functions

- [Enemy](#) (std::string texturePath, int type, std::shared_ptr< [Node](#) > n, int hitpoints, int speed)
Construct a new [Enemy](#) object.
- virtual [~Enemy](#) ()
Destroy the [Enemy](#) object.
- sf::Sprite & [getSprite](#) ()
Get the Sprite object.
- void [kill](#) ()
kill the enemy
- void [move](#) ()
Move the enemy.
- bool [isalive](#) ()
check if enemy is alive
- sf::FloatRect & [getHitbox](#) ()
Get the Hitbox object.
- int [getHitpoints](#) ()
Get the Hitpoints amount.
- void [setHitpoints](#) (int hp)

- *Set the amount of hitpoints.*
- void `takeDamage` (const int &dmg)
- *Deal damage to an enemy.*
- int `getSpeed` ()
- *Get the speed of the enemy.*
- int `getType` ()
- *Get the Type.*
- bool `shouldPay` ()
- *function to check if enemy has payed out*
- void `getPoisoned` ()
- *Apply poison to enemy.*
- void `checkPoisonDamage` ()
- *Check for poison damage.*
- bool `isFrozen` ()
- *check if enemy is currently frozen*
- void `Freeze` ()
- *freeze the enemy object*
- void `unFreeze` ()
- *unfreeze the enemy object*
- void `checkFreeze` ()
- *check if enemy is frozen*
- bool `atCheckpoint` ()
- *check if enemy has reached its next checkpoint*
- bool `setCheckpoint` ()
- *Set the next checkpoint of the enemy.*
- int `getSteps` ()
- *Get the amount of steps taken by enemy object.*

Protected Attributes

- int `hitpoints_`
- int `speed_`
- int `steps_taken_`
- bool `frozen_`
- int `freeze_timer_` = 0
- int `poisoned_` = 0
- int `poison_timer_` = 0

Private Attributes

- bool `x_goal_`
- bool `y_goal_`
- sf::Texture `texture_`
- sf::Sprite `sprite_`
- bool `has_payed_` = false
- bool `alive_`
- int `type_`
- sf::FloatRect `hitbox_`
- std::shared_ptr< Node > `next_point_`
- int `delta_x`
- int `delta_y`
- sf::SoundBuffer `death_sound_buffer_`
- sf::Sound `death_sound_`

5.8.1 Detailed Description

[Enemy](#) parent class.

5.8.2 Constructor & Destructor Documentation

5.8.2.1 Enemy()

```
Enemy::Enemy (
    std::string texturePath,
    int type,
    std::shared_ptr< Node > n,
    int hitpoints,
    int speed )
```

Construct a new [Enemy](#) object.

Parameters

<i>texturePath</i>	path to the texture
<i>type</i>	type of enemy
<i>n</i>	Node to begin from
<i>hitpoints</i>	amount of hitpoints
<i>speed</i>	

5.8.2.2 ~Enemy()

```
Enemy::~~Enemy ( ) [virtual]
```

Destroy the [Enemy](#) object.

5.8.3 Member Function Documentation

5.8.3.1 atCheckpoint()

```
bool Enemy::atCheckpoint ( )
```

check if enemy has reached its next checkpoint

Returns

true
false

5.8.3.2 checkFreeze()

```
void Enemy::checkFreeze ( )
```

check if enemy is frozen

5.8.3.3 checkPoisonDamage()

```
void Enemy::checkPoisonDamage ( )
```

Check for poison damage.

5.8.3.4 Freeze()

```
void Enemy::Freeze ( )
```

freeze the enemy object

5.8.3.5 getHitbox()

```
sf::FloatRect & Enemy::getHitbox ( )
```

Get the Hitbox object.

Returns

sf::FloatRect&

5.8.3.6 getHitpoints()

```
int Enemy::getHitpoints ( )
```

Get the Hitpoints amount.

Returns

int

5.8.3.7 getPoisoned()

```
void Enemy::getPoisoned ( )
```

Apply poison to enemy.

5.8.3.8 getSpeed()

```
int Enemy::getSpeed ( )
```

Get the speed of the enemy.

Returns

int speed

5.8.3.9 getSprite()

```
sf::Sprite & Enemy::getSprite ( )
```

Get the Sprite object.

Returns

sf::Sprite&

5.8.3.10 getSteps()

```
int Enemy::getSteps ( )
```

Get the amount of steps taken by enemy object.

Returns

int

5.8.3.11 getType()

```
int Enemy::getType ( )
```

Get the Type.

Returns

int type

5.8.3.12 isalive()

```
bool Enemy::isalive ( )
```

check if enemy is alive

Returns

true if alive

false otherwise

5.8.3.13 isFrozen()

```
bool Enemy::isFrozen ( )
```

check if enemy is currently frozen

Returns

true

false

5.8.3.14 kill()

```
void Enemy::kill ( )
```

kill the enemy

5.8.3.15 move()

```
void Enemy::move ( )
```

Move the enemy.

5.8.3.16 setCheckpoint()

```
bool Enemy::setCheckpoint ( )
```

Set the next checkpoint of the enemy.

Returns

true if finished is reached
false otherwise

5.8.3.17 setHitpoints()

```
void Enemy::setHitpoints (
    int hp )
```

Set the amount of hitpoints.

Parameters

<i>hp</i>	new hitpoint amount
-----------	---------------------

5.8.3.18 shouldPay()

```
bool Enemy::shouldPay ( )
```

function to check if enemy has payed out

Returns

true if enemy has no payed out
false otherwise

5.8.3.19 takeDamage()

```
void Enemy::takeDamage (
    const int & dmg )
```

Deal damage to an enemy.

Parameters

<i>dmg</i>	
------------	--

5.8.3.20 unFreeze()

```
void Enemy::unFreeze ( )
```

unfreeze the enemy object

5.8.4 Member Data Documentation**5.8.4.1 alive_**

```
bool Enemy::alive_ [private]
```

5.8.4.2 death_sound_

```
sf::Sound Enemy::death_sound_ [private]
```

5.8.4.3 death_sound_buffer_

```
sf::SoundBuffer Enemy::death_sound_buffer_ [private]
```

5.8.4.4 delta_x

```
int Enemy::delta_x [private]
```

5.8.4.5 delta_y

```
int Enemy::delta_y [private]
```


5.8.4.6 freeze_timer_

```
int Enemy::freeze_timer_ = 0 [protected]
```

5.8.4.7 frozen_

```
bool Enemy::frozen_ [protected]
```

5.8.4.8 has_payed_

```
bool Enemy::has_payed_ = false [private]
```

5.8.4.9 hitbox_

```
sf::FloatRect Enemy::hitbox_ [private]
```

5.8.4.10 hitpoints_

```
int Enemy::hitpoints_ [protected]
```

5.8.4.11 next_point_

```
std::shared_ptr<Node> Enemy::next_point_ [private]
```

5.8.4.12 poison_timer_

```
int Enemy::poison_timer_ = 0 [protected]
```

5.8.4.13 poisoned_

```
int Enemy::poisoned_ = 0 [protected]
```

5.8.4.14 speed_

```
int Enemy::speed_ [protected]
```

5.8.4.15 sprite_

```
sf::Sprite Enemy::sprite_ [private]
```

5.8.4.16 steps_taken_

```
int Enemy::steps_taken_ [protected]
```

5.8.4.17 texture_

```
sf::Texture Enemy::texture_ [private]
```

5.8.4.18 type_

```
int Enemy::type_ [private]
```

5.8.4.19 x_goal_

```
bool Enemy::x_goal_ [private]
```

5.8.4.20 y_goal_

```
bool Enemy::y_goal_ [private]
```

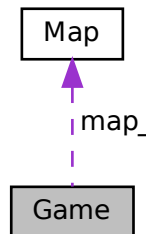
The documentation for this class was generated from the following files:

- [src/Enemy.hpp](#)
- [src/Enemy.cpp](#)

5.9 Game Class Reference

```
#include <Game.hpp>
```

Collaboration diagram for Game:



Public Member Functions

- **Game** (int *state)
Default game constructor.
- virtual **~Game** ()
Default game destructor.
- void **update** ()
Updates the game on the screen.
- void **render** ()
Renders all objects on the screen.
- bool **running** ()
Tells whether the game is running.
- bool **intersects** (sf::CircleShape &c, sf::FloatRect &r)
Function for checking if a circle and a rectangle intersects, from <https://stackoverflow.com/questions/401847/circle-rectangle-collision-detection-intersection>.

Private Member Functions

- void **pollEv** ()
- void **showErrorMessage** (const std::string &message)
Shows an error message.
- bool **checkPositions** ()
Check if enemies are within range.
- void **checkTowers** (sf::Vector2i mouse_location)
Check if any towers are clicked.
- void **checkButtons** (sf::Vector2i mouse_location)
Check if any button is clicked.
- void **showInfo** (unsigned int wave_nr, unsigned int info_type)
Show info window.
- void **moveTower** (std::shared_ptr< **Tower** > &tower, const sf::Vector2i &mouse_location)
Move a tower.
- bool **validPlacement** (sf::Vector2i mouse_location)
Checks if a tower can be placed at passed coordinate.

Private Attributes

- int * `state_`
- bool `placing_new_tower`
- bool `paused_`
- int `wave_number_`
- std::unique_ptr< [Player](#) > `player_`
- sf::RenderWindow * `window_`
- sf::Event `event_`
- std::vector< std::shared_ptr< [Enemy](#) > > `enemies_`
- std::vector< std::shared_ptr< [Tower](#) > > `towers_`
- std::vector< std::shared_ptr< [Node](#) > > `nodes_`
- [Map](#) `map_` = [Map](#)("assets/map.txt")
- std::vector< std::shared_ptr< [GameCell](#) > > `tiles_`
- std::vector< std::shared_ptr< [Button](#) > > `buttons_`
- sf::RectangleShape `button_background_`
- sf::Clock `clock`

5.9.1 Detailed Description

[Game](#) class

5.9.2 Constructor & Destructor Documentation

5.9.2.1 Game()

```
Game::Game (
    int * state )
```

Default game constructor.

5.9.2.2 ~Game()

```
Game::~~Game ( ) [virtual]
```

Default game destructor.

5.9.3 Member Function Documentation

5.9.3.1 checkButtons()

```
void Game::checkButtons (
    sf::Vector2i mouse_location ) [private]
```

Check if any button is clicked.

Parameters

<i>mouse_location</i>	Coordinates of mouse
-----------------------	----------------------

5.9.3.2 checkPositions()

```
bool Game::checkPositions ( ) [private]
```

Check if enemies are within range.

Returns

true if all enemies are dead
false if enemies are still alive

5.9.3.3 checkTowers()

```
void Game::checkTowers (
    sf::Vector2i mouse_location ) [private]
```

Check if any towers are clicked.

Parameters

<i>mouse_location</i>	Mouse coordinates
-----------------------	-------------------

5.9.3.4 intersects()

```
bool Game::intersects (
    sf::CircleShape & c,
    sf::FloatRect & r )
```

Function for checking if a circle and a rectangle intersects, from <https://stackoverflow.com/questions/401847/circle-rectangle-collision-detection-intersection>.

Parameters

<i>c</i>	the circle that should be checked
<i>r</i>	the rectangle to check

Returns

true if the shapes intersect
false if the shapes do not intersect

5.9.3.5 moveTower()

```
void Game::moveTower (
    std::shared_ptr< Tower > & tower,
    const sf::Vector2i & mouse_location ) [private]
```

Move a tower.

Parameters

<i>tower</i>	The tower to move
<i>mouse_location</i>	Coordinates of mouse

5.9.3.6 pollEv()

```
void Game::pollEv ( ) [private]
```

5.9.3.7 render()

```
void Game::render ( )
```

Renders all objects on the screen.

5.9.3.8 running()

```
bool Game::running ( )
```

Tells whether the game is running.

Returns

true if game is running
false if game is not running

5.9.3.9 showErrorMessage()

```
void Game::showErrorMessage (
    const std::string & message ) [private]
```

Shows an error message.

Parameters

<i>message</i>	Error message to be displayed
----------------	-------------------------------

5.9.3.10 showInfo()

```
void Game::showInfo (
    unsigned int wave_nr,
    unsigned int info_type ) [private]
```

Show info window.

Parameters

<i>mouse_location</i>	Coordinates of mouse
-----------------------	----------------------

5.9.3.11 update()

```
void Game::update ( )
```

Updates the game on the screen.

5.9.3.12 validPlacement()

```
bool Game::validPlacement (
    sf::Vector2i mouse_location ) [private]
```

Checks if a tower can be placed at passed coordinate.

Parameters

<i>mouse_location</i>	Coordinates of mouse
-----------------------	----------------------

Returns

true if tower can be placed
false if tower can't be placed

5.9.4 Member Data Documentation

5.9.4.1 button_background_

```
sf::RectangleShape Game::button_background_ [private]
```

5.9.4.2 buttons_

```
std::vector<std::shared_ptr<Button> > Game::buttons_ [private]
```

5.9.4.3 clock

```
sf::Clock Game::clock [private]
```

5.9.4.4 enemies_

```
std::vector<std::shared_ptr<Enemy> > Game::enemies_ [private]
```

5.9.4.5 event_

```
sf::Event Game::event_ [private]
```

5.9.4.6 map_

```
Map Game::map_ = Map("assets/map.txt") [private]
```

5.9.4.7 nodes_

```
std::vector<std::shared_ptr<Node> > Game::nodes_ [private]
```

5.9.4.8 paused_

```
bool Game::paused_ [private]
```


5.9.4.9 placing_new_tower

```
bool Game::placing_new_tower [private]
```

5.9.4.10 player_

```
std::unique_ptr<Player> Game::player_ [private]
```

5.9.4.11 state_

```
int* Game::state_ [private]
```

5.9.4.12 tiles_

```
std::vector<std::shared_ptr<GameCell> > Game::tiles_ [private]
```

5.9.4.13 towers_

```
std::vector<std::shared_ptr<Tower> > Game::towers_ [private]
```

5.9.4.14 wave_number_

```
int Game::wave_number_ [private]
```

5.9.4.15 window_

```
sf::RenderWindow* Game::window_ [private]
```

The documentation for this class was generated from the following files:

- [src/Game.hpp](#)
- [src/Game.cpp](#)

5.10 GameCell Class Reference

```
#include <gameCell.hpp>
```

Public Member Functions

- [GameCell](#) (std::string path, float x, float y, bool passable, bool exit, bool start)
Constructor for initializing a [Map](#) object.
- [~GameCell](#) ()
[GameCell](#) object destructor.
- bool [setUpSprite](#) (std::string path)
Constructor for initializing a [Map](#) object.
- sf::Sprite & [getSprite](#) ()
Returns a reference to the sprite variable.

Public Attributes

- bool [isExit_](#)
- bool [isStart_](#)
- bool [isPassable_](#)
- sf::Vector2f [pos_](#)
- sf::Texture [texture_](#)
- sf::Sprite [sprite_](#)

5.10.1 Constructor & Destructor Documentation

5.10.1.1 GameCell()

```
GameCell::GameCell (
    std::string path,
    float x,
    float y,
    bool passable,
    bool exit,
    bool start )
```

Constructor for initializing a [Map](#) object.

Parameters

<i>path</i>	Path to sprite asset
<i>x</i>	The desired x coordinate of the sprite
<i>y</i>	The desired y coordinate of the sprite
<i>passable</i>	Boolean value that tells if the cell is passable
<i>exit</i>	Boolean value to set the exit of the map
<i>start</i>	Boolean parameter to set the start of the map

5.10.1.2 ~GameCell()

```
GameCell::~~GameCell ( ) [inline]
```

[GameCell](#) object destructor.

5.10.2 Member Function Documentation

5.10.2.1 getSprite()

```
sf::Sprite & GameCell::getSprite ( )
```

Returns a reference to the sprite variable.

Returns

Reference to the sprite variable

5.10.2.2 setUpSprite()

```
bool GameCell::setUpSprite (
    std::string path )
```

Constructor for initializing a [Map](#) object.

Parameters

<i>path</i>	Path to sprite asset
-------------	----------------------

Returns

true if successful, false otherwise

5.10.3 Member Data Documentation

5.10.3.1 isExit_

```
bool GameCell::isExit_
```

5.10.3.2 isPassable_

```
bool GameCell::isPassable_
```

5.10.3.3 isStart_

```
bool GameCell::isStart_
```

5.10.3.4 pos_

```
sf::Vector2f GameCell::pos_
```

5.10.3.5 sprite_

```
sf::Sprite GameCell::sprite_
```

5.10.3.6 texture_

```
sf::Texture GameCell::texture_
```

The documentation for this class was generated from the following files:

- [src/gameCell.hpp](#)
- [src/gameCell.cpp](#)

5.11 GameOver Class Reference

Handles state after game finishes execution.

```
#include <Game_over.hpp>
```

Public Member Functions

- [GameOver](#) (int *state)
Construct a new [Game](#) Over object.
- bool [running](#) ()
If window is open.
- void [update](#) ()
Update the window state.
- void [render](#) ()
Render the elemets.

Private Member Functions

- void [checkButtons](#) (sf::Vector2i mouse_location)
Check if any of the buttons has been clicked.
- void [pollEv](#) ()
Check for events (mouse clicks, keyboard input etc.)

Private Attributes

- std::vector< std::shared_ptr< [Button](#) > > [buttons_](#)
- sf::Event [event_](#)
- sf::RenderWindow * [window_](#)
- int * [state_](#)
- sf::Text [text_](#)
- sf::Font [font_](#)

5.11.1 Detailed Description

Handles state after game finishes execution.

5.11.2 Constructor & Destructor Documentation

5.11.2.1 GameOver()

```
GameOver::GameOver (
    int * state )
```

Construct a new [Game](#) Over object.

Parameters

<i>state</i>	state when game finished execution
--------------	------------------------------------

5.11.3 Member Function Documentation

5.11.3.1 checkButtons()

```
void GameOver::checkButtons (
    sf::Vector2i mouse_location ) [private]
```

Check if any of the buttons has been clicked.

Parameters

<i>mouse_location</i>	
-----------------------	--

5.11.3.2 pollEv()

```
void GameOver::pollEv ( ) [private]
```

Check for events (mouse clicks, keyboard input etc.)

5.11.3.3 render()

```
void GameOver::render ( )
```

Render the elemets.

5.11.3.4 running()

```
bool GameOver::running ( )
```

If window is open.

Returns

true

false

5.11.3.5 update()

```
void GameOver::update ( )
```

Update the window state.

5.11.4 Member Data Documentation

5.11.4.1 buttons_

```
std::vector<std::shared_ptr<Button> > GameOver::buttons_ [private]
```

5.11.4.2 event_

```
sf::Event GameOver::event_ [private]
```

5.11.4.3 font_

```
sf::Font GameOver::font_ [private]
```

5.11.4.4 state_

```
int* GameOver::state_ [private]
```

5.11.4.5 text_

```
sf::Text GameOver::text_ [private]
```

5.11.4.6 window_

```
sf::RenderWindow* GameOver::window_ [private]
```

The documentation for this class was generated from the following files:

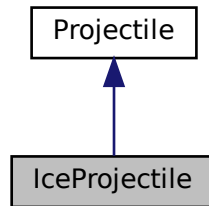
- [src/Game_over.hpp](#)
- [src/Game_over.cpp](#)

5.12 IceProjectile Class Reference

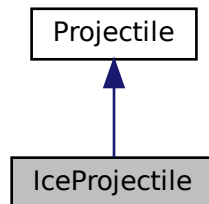
Ice projectile for ice tower.

```
#include <Ice_projectile.hpp>
```

Inheritance diagram for IceProjectile:



Collaboration diagram for IceProjectile:



Public Member Functions

- [IceProjectile](#) (std::shared_ptr< [Enemy](#) > enemy, sf::Vector2f origin)
Construct a new Ice [Projectile](#) object.

Additional Inherited Members

5.12.1 Detailed Description

Ice projectile for ice tower.

5.12.2 Constructor & Destructor Documentation

5.12.2.1 IceProjectile()

```
IceProjectile::IceProjectile (
    std::shared_ptr< Enemy > enemy,
    sf::Vector2f origin )
```

Construct a new Ice [Projectile](#) object.

Parameters

<i>enemy</i>	target
<i>origin</i>	position of tower

The documentation for this class was generated from the following files:

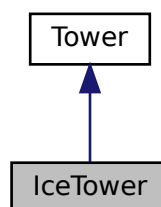
- [src/Ice_projectile.hpp](#)
- [src/Ice_projectile.cpp](#)

5.13 IceTower Class Reference

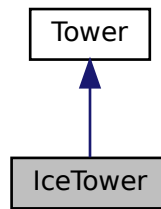
Ice tower class.

```
#include <Ice_tower.hpp>
```

Inheritance diagram for IceTower:



Collaboration diagram for IceTower:



Public Member Functions

- `IceTower ()`
Construct a new Ice [Tower](#) object.
- `~IceTower ()`
Destroy the Ice [Tower](#) object.
- `void fire ()`
Fire at enemy.
- `std::vector< std::shared_ptr< Projectile > > & getProjectiles ()`
Get the [Projectiles](#) object.

Private Attributes

- `std::vector< std::shared_ptr< Projectile > > projectiles_`

Additional Inherited Members

5.13.1 Detailed Description

Ice tower class.

5.13.2 Constructor & Destructor Documentation

5.13.2.1 IceTower()

```
IceTower::IceTower ( )
```

Construct a new Ice [Tower](#) object.

5.13.2.2 ~IceTower()

```
IceTower::~~IceTower ( )
```

Destroy the Ice [Tower](#) object.

5.13.3 Member Function Documentation

5.13.3.1 fire()

```
void IceTower::fire ( ) [virtual]
```

Fire at enemy.

Implements [Tower](#).

5.13.3.2 getProjectiles()

```
std::vector< std::shared_ptr< Projectile > > & IceTower::getProjectiles ( ) [virtual]
```

Get the Projectiles object.

Returns

```
std::vector<std::shared_ptr<Projectile>>&
```

Implements [Tower](#).

5.13.4 Member Data Documentation

5.13.4.1 projectiles_

```
std::vector<std::shared_ptr<Projectile> > IceTower::projectiles_ [private]
```

The documentation for this class was generated from the following files:

- [src/lce_tower.hpp](#)
- [src/lce_tower.cpp](#)

5.14 mainMenu Class Reference

```
#include <mainMenu.hpp>
```

Public Member Functions

- [mainMenu](#) ()
Constructor for initializing a [mainMenu](#) object.
- [~mainMenu](#) ()
Destructor for a [mainMenu](#) object.
- void [Render](#) ()
Draws the main menu window. Takes no parameters and returns nothing.
- void [Update](#) ()
Updates the main menu window frame by frame.
- void [pollEv](#) ()
Checks for events in the main menu window.
- bool [isRunning](#) ()
Checks if the window is still open returns a boolean value.
- int [getNextWindow](#) ()
Returns an integer that defines which window to open after main menu.

Private Attributes

- sf::Text [gameTitle_](#)
- sf::Text [playButton_](#)
- sf::Text [mapButton_](#)
- sf::Text [exitButton_](#)
- sf::RenderWindow * [window_](#)
- sf::Event [event_](#)
- sf::Font [font_](#)
- std::vector< std::shared_ptr< [Button](#) > > [buttons_](#)
- bool [playButtonSelected_](#)
- bool [playButtonPressed_](#)
- bool [exitButtonSelected_](#)
- bool [exitButtonPressed_](#)
- bool [mapButtonSelected_](#)
- bool [mapButtonPressed_](#)
- int [next_window_](#)

5.14.1 Constructor & Destructor Documentation

5.14.1.1 mainMenu()

```
mainMenu::mainMenu ( )
```

Constructor for initializing a [mainMenu](#) object.

5.14.1.2 ~mainMenu()

```
mainMenu::~~mainMenu ( ) [inline]
```

Destructor for a [mainMenu](#) object.

5.14.2 Member Function Documentation

5.14.2.1 getNextWindow()

```
int mainMenu::getNextWindow ( )
```

Returns an integer that defines which window to open after main menu.

5.14.2.2 isRunning()

```
bool mainMenu::isRunning ( )
```

Checks if the window is still open returns a boolean value.

5.14.2.3 pollEv()

```
void mainMenu::pollEv ( )
```

Checks for events in the main menu window.

5.14.2.4 Render()

```
void mainMenu::Render ( )
```

Draws the main menu window. Takes no parameters and returns nothing.

5.14.2.5 Update()

```
void mainMenu::Update ( )
```

Updates the main menu window frame by frame.

5.14.3 Member Data Documentation

5.14.3.1 buttons_

```
std::vector<std::shared_ptr<Button> > mainMenu::buttons_ [private]
```

5.14.3.2 event_

```
sf::Event mainMenu::event_ [private]
```

5.14.3.3 exitButton_

```
sf::Text mainMenu::exitButton_ [private]
```

5.14.3.4 exitButtonPressed_

```
bool mainMenu::exitButtonPressed_ [private]
```

5.14.3.5 exitButtonSelected_

```
bool mainMenu::exitButtonSelected_ [private]
```

5.14.3.6 font_

```
sf::Font mainMenu::font_ [private]
```

5.14.3.7 gameTitle_

```
sf::Text mainMenu::gameTitle_ [private]
```

5.14.3.8 mapButton_

```
sf::Text mainMenu::mapButton_ [private]
```

5.14.3.9 mapButtonPressed_

```
bool mainMenu::mapButtonPressed_ [private]
```

5.14.3.10 mapButtonSelected_

```
bool mainMenu::mapButtonSelected_ [private]
```

5.14.3.11 next_window_

```
int mainMenu::next_window_ [private]
```

5.14.3.12 playButton_

```
sf::Text mainMenu::playButton_ [private]
```

5.14.3.13 playButtonPressed_

```
bool mainMenu::playButtonPressed_ [private]
```

5.14.3.14 playButtonSelected_

```
bool mainMenu::playButtonSelected_ [private]
```

5.14.3.15 window_

```
sf::RenderWindow* mainMenu::window_ [private]
```

The documentation for this class was generated from the following files:

- [src/mainMenu.hpp](#)
- [src/mainMenu.cpp](#)

5.15 Map Class Reference

```
#include <Map.hpp>
```

Public Member Functions

- [Map](#) (std::string path)
Constructor for initializing a [Map](#) object.
- [~Map](#) ()
[Map](#) object destructor.
- void [setUpCells](#) ()
Sets up a matrix of tiles that are used to render the map. Takes no parameters and return nothing.
- std::vector< std::shared_ptr< [Node](#) > > [getNodes](#) () const
Returns a vector of shared pointers to [Node](#) objects Takes no parameters.
- std::vector< std::vector< std::shared_ptr< [GameCell](#) > > > [getTiles](#) () const
Returns a matrix of shared pointers to [GameCell](#) objects Takes no parameters.

Private Attributes

- std::vector< std::vector< std::shared_ptr< [GameCell](#) > > > [tile_matrix_](#)
- std::string [file_path_](#)
- std::vector< std::shared_ptr< [Node](#) > > [nodes_](#)

5.15.1 Constructor & Destructor Documentation

5.15.1.1 Map()

```
Map::Map (
    std::string path )
```

Constructor for initializing a [Map](#) object.

Parameters

<i>filename</i>	Path to file where the map is read from
-----------------	---

5.15.1.2 ~Map()

```
Map::~~Map ( ) [inline]
```

[Map](#) object destructor.

5.15.2 Member Function Documentation

5.15.2.1 getNodes()

```
std::vector< std::shared_ptr< Node > > Map::getNodes ( ) const
```

Returns a vector of shared pointers to [Node](#) objects Takes no parameters.

Returns

Vector of shared pointers to [Node](#) objects

5.15.2.2 getTiles()

```
std::vector< std::vector< std::shared_ptr< GameCell > > > Map::getTiles ( ) const
```

Returns a matrix of shared pointers to [GameCell](#) objects Takes no parameters.

Returns

Matrix of shared pointers to [GameCell](#) objects

5.15.2.3 setUpCells()

```
void Map::setUpCells ( )
```

Sets up a matrix of tiles that are used to render the map. Takes no parameters and return nothing.

5.15.3 Member Data Documentation

5.15.3.1 file_path_

```
std::string Map::file_path_ [private]
```

5.15.3.2 nodes_

```
std::vector<std::shared_ptr<Node> > Map::nodes_ [private]
```

5.15.3.3 tile_matrix_

```
std::vector<std::vector<std::shared_ptr<GameCell> > > Map::tile_matrix_ [private]
```

The documentation for this class was generated from the following files:

- [src/Map.hpp](#)
- [src/Map.cpp](#)

5.16 mapCreator Class Reference

```
#include <mapCreator.hpp>
```

Public Member Functions

- [mapCreator](#) ()
Constructor for initializing a [mapCreator](#) object.
- [~mapCreator](#) ()
[mapCreator](#) object destructor
- void [Render](#) ()
Draws the map creator window. Takes no parameters and returns nothing.
- void [Update](#) ()
Updates the map creator window frame by frame.
- void [pollEv](#) ()
Checks for events in the map creator window.
- void [checkButtons](#) (sf::Vector2i mouseLocation)
Checks if a button or a game tile gets pressed in the map creator window.
- bool [isRunning](#) ()
Checks if the map creator window is still open returns a boolean value.
- void [showInfo](#) (unsigned int info_type)
Creates and displays an info window.

Private Attributes

- sf::RenderWindow * [window_](#)
- sf::Event [event_](#)
- std::vector< std::vector< std::shared_ptr< [Button](#) > > > [buttons_](#)
- std::shared_ptr< [Button](#) > [saveButton_](#)
- std::shared_ptr< [Button](#) > [mainMenuButton_](#)
- std::shared_ptr< [Button](#) > [setStartButton_](#)
- std::shared_ptr< [Button](#) > [setFinnishButton_](#)

5.16.1 Constructor & Destructor Documentation

5.16.1.1 mapCreator()

```
mapCreator::mapCreator ( )
```

Constructor for initializing a [mapCreator](#) object.

5.16.1.2 ~mapCreator()

```
mapCreator::~mapCreator ( )
```

[mapCreator](#) object destructor

5.16.2 Member Function Documentation

5.16.2.1 checkButtons()

```
void mapCreator::checkButtons (
    sf::Vector2i mouseLocation )
```

Checks if a button or a game tile gets pressed in the map creator window.

5.16.2.2 isRunning()

```
bool mapCreator::isRunning ( )
```

Checks if the map creator window is still open returns a boolean value.

5.16.2.3 pollEv()

```
void mapCreator::pollEv ( )
```

Checks for events in the map creator window.

5.16.2.4 Render()

```
void mapCreator::Render ( )
```

Draws the map creator window. Takes no parameters and returns nothing.

5.16.2.5 showInfo()

```
void mapCreator::showInfo (
    unsigned int info_type )
```

Creates and displays an info window.

5.16.2.6 Update()

```
void mapCreator::Update ( )
```

Updates the map creator window frame by frame.

5.16.3 Member Data Documentation

5.16.3.1 buttons_

```
std::vector<std::vector<std::shared_ptr<Button> > > mapCreator::buttons_ [private]
```

5.16.3.2 event_

```
sf::Event mapCreator::event_ [private]
```

5.16.3.3 mainMenuButton_

```
std::shared_ptr<Button> mapCreator::mainMenuButton_ [private]
```

5.16.3.4 saveButton_

```
std::shared_ptr<Button> mapCreator::saveButton_ [private]
```

5.16.3.5 setFinnishButton_

```
std::shared_ptr<Button> mapCreator::setFinnishButton_ [private]
```

5.16.3.6 setStartButton_

```
std::shared_ptr<Button> mapCreator::setStartButton_ [private]
```

5.16.3.7 window_

```
sf::RenderWindow* mapCreator::window_ [private]
```

The documentation for this class was generated from the following files:

- [src/mapCreator.hpp](#)
- [src/mapCreator.cpp](#)

5.17 Node Class Reference

```
#include <Node.hpp>
```

Public Member Functions

- [Node](#) (std::shared_ptr< [Node](#) > ptr=NULL, float x=0, float y=0)
- sf::Vector2f & [get_cord](#) ()
- std::shared_ptr< [Node](#) > [get_next](#) ()
- void [set_cord](#) (sf::Vector2f &v)
Set the cord object.
- void [set_next](#) (std::shared_ptr< [Node](#) > n)
Set the next object.

Private Attributes

- `std::shared_ptr< Node > next_`
- `sf::Vector2f coordinate\_`

5.17.1 Constructor & Destructor Documentation

5.17.1.1 Node()

```
Node::Node (
    std::shared_ptr< Node > ptr = NULL,
    float x = 0,
    float y = 0 )
```

5.17.2 Member Function Documentation

5.17.2.1 get_cord()

```
sf::Vector2f & Node::get_cord ( )
```

5.17.2.2 get_next()

```
std::shared_ptr< Node > Node::get_next ( )
```

5.17.2.3 set_cord()

```
void Node::set_cord (
    sf::Vector2f & v )
```

Set the cord object.

Parameters

<i>v</i>	coordinate
----------	------------

5.17.2.4 set_next()

```
void Node::set_next (
    std::shared_ptr< Node > n )
```

Set the next object.

Parameters

<i>n</i>	next object
----------	-------------

5.17.3 Member Data Documentation

5.17.3.1 coordinate_

```
sf::Vector2f Node::coordinate_ [private]
```

5.17.3.2 next_

```
std::shared_ptr<Node> Node::next_ [private]
```

The documentation for this class was generated from the following files:

- [src/Node.hpp](#)
- [src/Node.cpp](#)

5.18 Player Class Reference

Class for handling gamestate from player perspective.

```
#include <Player.hpp>
```

Public Member Functions

- [Player](#) (int hp=0, int money=0)
Construct a new [Player](#) object.
- void [giveMoney](#) (int amount)
Add money to player.
- bool [takeMoney](#) (int amount)
Remove money from player.
- void [heal](#) (int amount)
Add health to player.
- bool [takeDamage](#) (int amount)
remove health from player
- int [getHealth](#) ()
Get the Health amount.
- int [getMoney](#) ()
Get the Money amount.
- sf::Text & [getHealthText](#) ()
Get the Health Text object.
- sf::Text & [getMoneyText](#) ()
Get the Money Text object.

Private Attributes

- int [health_](#)
- int [money_](#)
- sf::Text [hp_text_](#)
- sf::Text [money_text_](#)
- sf::Font [font_](#)

5.18.1 Detailed Description

Class for handling gamestate from player perspective.

5.18.2 Constructor & Destructor Documentation

5.18.2.1 Player()

```
Player::Player (
    int hp = 0,
    int money = 0 )
```

Construct a new [Player](#) object.

Parameters

<i>hp</i>	starting health
<i>money</i>	starting money

5.18.3 Member Function Documentation

5.18.3.1 `getHealth()`

```
int Player::getHealth ( )
```

Get the Health amount.

Returns

int amount of health

5.18.3.2 `getHealthText()`

```
sf::Text & Player::getHealthText ( )
```

Get the Health Text object.

Returns

sf::Text&

5.18.3.3 `getMoney()`

```
int Player::getMoney ( )
```

Get the Money amount.

Returns

int amount of money

5.18.3.4 `getMoneyText()`

```
sf::Text & Player::getMoneyText ( )
```

Get the Money Text object.

Returns

sf::Text&

5.18.3.5 `giveMoney()`

```
void Player::giveMoney (
    int amount )
```

Add money to player.

Parameters

<i>amount</i>	amount to add
---------------	---------------

5.18.3.6 heal()

```
void Player::heal (  
    int amount )
```

Add health to player.

Parameters

<i>amount</i>	amount to heal
---------------	----------------

5.18.3.7 takeDamage()

```
bool Player::takeDamage (  
    int amount )
```

remove health from player

Parameters

<i>amount</i>	amount to remove
---------------	------------------

Returns

true

false

5.18.3.8 takeMoney()

```
bool Player::takeMoney (  
    int amount )
```

Remove money from player.

Parameters

<i>amount</i>	to remove
---------------	-----------

Returns

true if player has enough money
false if player does not have enough money

5.18.4 Member Data Documentation

5.18.4.1 font_

```
sf::Font Player::font_ [private]
```

5.18.4.2 health_

```
int Player::health_ [private]
```

5.18.4.3 hp_text_

```
sf::Text Player::hp_text_ [private]
```

5.18.4.4 money_

```
int Player::money_ [private]
```

5.18.4.5 money_text_

```
sf::Text Player::money_text_ [private]
```

The documentation for this class was generated from the following files:

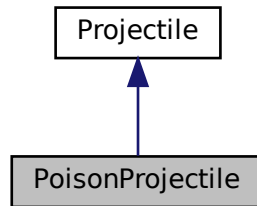
- [src/Player.hpp](#)
- [src/Player.cpp](#)

5.19 PoisonProjectile Class Reference

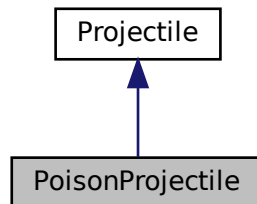
Poison projectile for poison tower.

```
#include <Poison_projectile.hpp>
```

Inheritance diagram for PoisonProjectile:



Collaboration diagram for PoisonProjectile:



Public Member Functions

- `PoisonProjectile` (`std::shared_ptr< Enemy >` enemy, `sf::Vector2f` origin)
Construct a new [Poison Projectile](#) object.

Additional Inherited Members

5.19.1 Detailed Description

Poison projectile for poison tower.

5.19.2 Constructor & Destructor Documentation

5.19.2.1 PoisonProjectile()

```
PoisonProjectile::PoisonProjectile (
    std::shared_ptr< Enemy > enemy,
    sf::Vector2f origin )
```

Construct a new Poison [Projectile](#) object.

Parameters

<i>enemy</i>	target
<i>origin</i>	position of tower

The documentation for this class was generated from the following files:

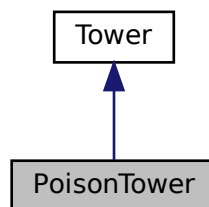
- [src/Poison_projectile.hpp](#)
- [src/Poison_projectile.cpp](#)

5.20 PoisonTower Class Reference

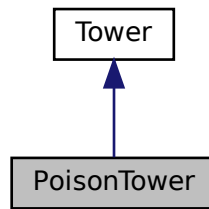
Poison tower class.

```
#include <Poison_tower.hpp>
```

Inheritance diagram for PoisonTower:



Collaboration diagram for PoisonTower:



Public Member Functions

- `PoisonTower ()`
Construct a new Poison [Tower](#) object.
- `~PoisonTower ()`
Destroy the Poison [Tower](#) object.
- `void fire ()`
Fire at enemy.
- `std::vector< std::shared_ptr< Projectile > > & getProjectiles ()`
Get the [Projectiles](#) object.

Private Attributes

- `std::vector< std::shared_ptr< Projectile > > projectiles_`

Additional Inherited Members

5.20.1 Detailed Description

Poison tower class.

5.20.2 Constructor & Destructor Documentation

5.20.2.1 PoisonTower()

```
PoisonTower::PoisonTower ( )
```

Construct a new Poison [Tower](#) object.

5.20.2.2 ~PoisonTower()

```
PoisonTower::~~PoisonTower ( )
```

Destroy the Poison [Tower](#) object.

5.20.3 Member Function Documentation

5.20.3.1 fire()

```
void PoisonTower::fire ( ) [virtual]
```

Fire at enemy.

Implements [Tower](#).

5.20.3.2 getProjectiles()

```
std::vector< std::shared_ptr< Projectile > > & PoisonTower::getProjectiles ( ) [virtual]
```

Get the Projectiles object.

Returns

```
std::vector<std::shared_ptr<Projectile>>&
```

Implements [Tower](#).

5.20.4 Member Data Documentation

5.20.4.1 projectiles_

```
std::vector<std::shared_ptr<Projectile> > PoisonTower::projectiles_ [private]
```

The documentation for this class was generated from the following files:

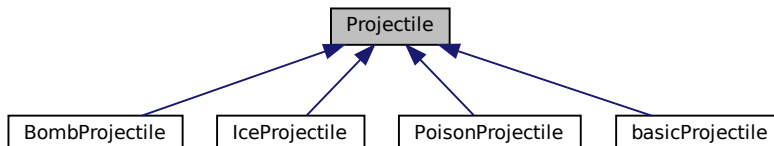
- [src/Poison_tower.hpp](#)
- [src/Poison_tower.cpp](#)

5.21 Projectile Class Reference

Parent class for projectiles.

```
#include <Projectile.hpp>
```

Inheritance diagram for Projectile:



Public Member Functions

- [Projectile](#) (std::string texturePath, int type, std::shared_ptr< [Enemy](#) > enemy, const sf::Vector2f origin)
Construct a new [Projectile](#) object.
- virtual [~Projectile](#) ()
Destroy the [Projectile](#) object.
- sf::Sprite & [getSprite](#) ()
return the projectile sprite
- sf::FloatRect & [getHitbox](#) ()
return the projectile hitbox
- std::shared_ptr< [Enemy](#) > [getEnemy](#) ()
return the projectile target
- bool [move](#) ()
Move the projectile.
- int [getDamage](#) () const
Return the damage of the projectile.

Protected Attributes

- int [damage_](#)

Private Attributes

- sf::Texture [texture_](#)
- sf::Sprite [sprite_](#)
- sf::FloatRect [hitbox_](#)
- int [type_](#)
- int [speed_](#) = 8
- std::shared_ptr< [Enemy](#) > [enemy_](#)

5.21.1 Detailed Description

Parent class for projectiles.

5.21.2 Constructor & Destructor Documentation

5.21.2.1 Projectile()

```
Projectile::Projectile (
    std::string texturePath,
    int type,
    std::shared_ptr< Enemy > enemy,
    const sf::Vector2f origin )
```

Construct a new [Projectile](#) object.

Parameters

<i>texturePath</i>	Path to the projectile texture
<i>type</i>	type of projectile
<i>enemy</i>	target for the projectile
<i>origin</i>	the starting point of the projectile

5.21.2.2 ~Projectile()

```
Projectile::~~Projectile ( ) [virtual]
```

Destroy the [Projectile](#) object.

5.21.3 Member Function Documentation

5.21.3.1 getDamage()

```
int Projectile::getDamage ( ) const
```

Return the damage of the projectile.

Returns

int

5.21.3.2 getEnemy()

```
std::shared_ptr< Enemy > Projectile::getEnemy ( )
```

return the projectile target

Returns

std::shared_ptr<Enemy>

5.21.3.3 getHitbox()

```
sf::FloatRect & Projectile::getHitbox ( )
```

return the projectile hitbox

Returns

sf::FloatRect&

5.21.3.4 getSprite()

```
sf::Sprite & Projectile::getSprite ( )
```

return the projectile sprite

Returns

sf::Sprite&

5.21.3.5 move()

```
bool Projectile::move ( )
```

Move the projectile.

Returns

true when intersecting with enemy
false otherwise

5.21.4 Member Data Documentation

5.21.4.1 damage_

```
int Projectile::damage_ [protected]
```

5.21.4.2 enemy_

```
std::shared_ptr<Enemy> Projectile::enemy_ [private]
```

5.21.4.3 hitbox_

```
sf::FloatRect Projectile::hitbox_ [private]
```

5.21.4.4 speed_

```
int Projectile::speed_ = 8 [private]
```

5.21.4.5 sprite_

```
sf::Sprite Projectile::sprite_ [private]
```

5.21.4.6 texture_

```
sf::Texture Projectile::texture_ [private]
```

5.21.4.7 type_

```
int Projectile::type_ [private]
```

The documentation for this class was generated from the following files:

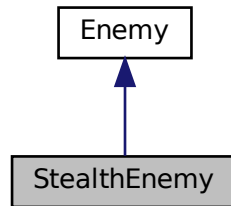
- [src/Projectile.hpp](#)
- [src/Projectile.cpp](#)

5.22 StealthEnemy Class Reference

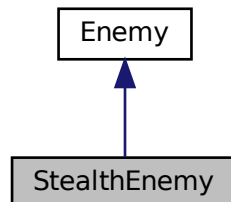
Child class for stealth enemies.

```
#include <Stealth_enemy.hpp>
```

Inheritance diagram for StealthEnemy:



Collaboration diagram for StealthEnemy:



Public Member Functions

- `StealthEnemy` (`std::shared_ptr< Node > n`)
Construct a new Stealth [Enemy](#) object.

Additional Inherited Members

5.22.1 Detailed Description

Child class for stealth enemies.

5.22.2 Constructor & Destructor Documentation

5.22.2.1 StealthEnemy()

```
StealthEnemy::StealthEnemy (
    std::shared_ptr< Node > n )
```

Construct a new Stealth [Enemy](#) object.

Parameters

<i>n</i>	starting node
----------	---------------

The documentation for this class was generated from the following files:

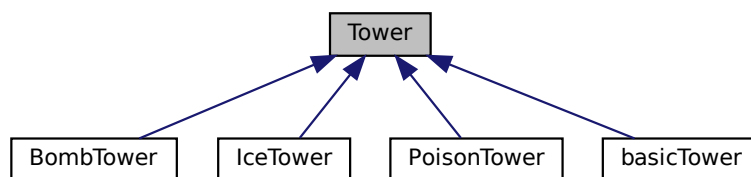
- [src/Stealth_enemy.hpp](#)
- [src/Stealth_enemy.cpp](#)

5.23 Tower Class Reference

Parent class for different towers.

```
#include <Tower.hpp>
```

Inheritance diagram for Tower:



Public Member Functions

- [Tower](#) (std::string texturePath, int type, float range, int attack_speed)
Tower constructor.
- virtual [~Tower](#) ()
Tower destructor.
- sf::Sprite & [getSprite](#) ()
Get the Sprite object.

- void `setSelected ()`
Set the object state as selected (true)
- void `resetSelected ()`
Set the object state as not selected (false)
- void `upgrade ()`
Upgrade the tower attack speed.
- bool `isSelected () const`
Check if the tower is selected.
- int `getRange () const`
Get the tower range.
- int `getAttackSpeed () const`
Get the tower attack speed.
- int `getType () const`
Get the tower type.
- int `getLevel () const`
Get the tower level.
- sf::CircleShape & `getRangeCircle ()`
Get the geometric circle that represents the tower range.
- void `setTarget (std::shared_ptr< Enemy > enemy)`
Set the tower's target.
- void `removeTarget ()`
Remove the tower's target.
- bool `hasTarget ()`
Check if tower has selected a target.
- std::shared_ptr< Enemy > `getTarget ()`
Returns the tower's current target.
- virtual void `fire ()=0`
Makes the tower fire a projectile at an enemy This is a virtual function since every tower type has its own projectile type.
- virtual std::vector< std::shared_ptr< Projectile > > & `getProjectiles ()=0`
Get the tower projectiles as a vector.
- void `setLastFireTime (sf::Time time)`
Set the time since last attack for tower.
- sf::Time `getLastFireTime () const`
Returns the time since last attack for tower.
- sf::Text & `getText ()`
Get the tower level text (graphical element)

Protected Attributes

- int `level_` = 1
- int `price_`
- float `range_`
- int `attack_speed_`
- bool `has_target_` = false
- std::shared_ptr< Enemy > `target_`
- sf::CircleShape `range_circle_`
- sf::Sprite `sprite_`

Private Attributes

- sf::Text [text_](#)
- sf::Font [font_](#)
- sf::Texture [texture_](#)
- bool [selected_](#)
- int [type_](#)
- sf::Time [last_fire_time_](#)

5.23.1 Detailed Description

Parent class for different towers.

5.23.2 Constructor & Destructor Documentation

5.23.2.1 Tower()

```
Tower::Tower (
    std::string texturePath,
    int type,
    float range,
    int attack_speed )
```

[Tower](#) constructor.

Parameters

<i>texturePath</i>	Path to the tower sprite.
<i>type</i>	Type of tower.
<i>range</i>	The towers range.
<i>attack_speed</i>	The towers attack speed.

5.23.2.2 ~Tower()

```
Tower::~~Tower ( ) [virtual]
```

[Tower](#) destructor.

5.23.3 Member Function Documentation

5.23.3.1 fire()

```
virtual void Tower::fire ( ) [pure virtual]
```

Makes the tower fire a projectile at an enemy This is a virtual function since every tower type has its own projectile type.

Implemented in [PoisonTower](#), [IceTower](#), [BombTower](#), and [basicTower](#).

5.23.3.2 getAttackSpeed()

```
int Tower::getAttackSpeed ( ) const
```

Get the tower attack speed.

Returns

int

5.23.3.3 getLastFireTime()

```
sf::Time Tower::getLastFireTime ( ) const
```

Returns the time since last attack for tower.

Returns

sf::Time

5.23.3.4 getLevel()

```
int Tower::getLevel ( ) const
```

Get the tower level.

Returns

int

5.23.3.5 getProjectiles()

```
virtual std::vector<std::shared_ptr<Projectile> >& Tower::getProjectiles ( ) [pure virtual]
```

Get the tower projectiles as a vector.

Returns

`std::vector<std::shared_ptr<Projectile>>&`

Implemented in [PoisonTower](#), [IceTower](#), [BombTower](#), and [basicTower](#).

5.23.3.6 getRange()

```
int Tower::getRange ( ) const
```

Get the tower range.

Returns

`int`

5.23.3.7 getRangeCircle()

```
sf::CircleShape & Tower::getRangeCircle ( )
```

Get the geometric circle that represents the tower range.

Returns

`sf::CircleShape&`

5.23.3.8 getSprite()

```
sf::Sprite & Tower::getSprite ( )
```

Get the Sprite object.

Returns

`sf::Sprite&`

5.23.3.9 `getTarget()`

```
std::shared_ptr< Enemy > Tower::getTarget ( )
```

Returns the tower's current target.

Returns

`std::shared_ptr<Enemy>`

5.23.3.10 `getText()`

```
sf::Text & Tower::getText ( )
```

Get the tower level text (graphical element)

Returns

`sf::Text&`

5.23.3.11 `getType()`

```
int Tower::getType ( ) const
```

Get the tower type.

Returns

`int`

5.23.3.12 `hasTarget()`

```
bool Tower::hasTarget ( )
```

Check if tower has selected a target.

Returns

`true`

`false`

5.23.3.13 isSelected()

```
bool Tower::isSelected ( ) const
```

Check if the tower is selected.

Returns

true
false

5.23.3.14 removeTarget()

```
void Tower::removeTarget ( )
```

Remove the tower's target.

5.23.3.15 resetSelected()

```
void Tower::resetSelected ( )
```

Set the object state as not selected (false)

5.23.3.16 setLastFireTime()

```
void Tower::setLastFireTime (
    sf::Time time )
```

Set the time since last attack for tower.

Parameters

<i>time</i>	
-------------	--

5.23.3.17 setSelected()

```
void Tower::setSelected ( )
```

Set the object state as selected (true)

5.23.3.18 setTarget()

```
void Tower::setTarget (
    std::shared_ptr< Enemy > enemy )
```

Set the tower's target.

Parameters

<i>enemy</i>	
--------------	--

5.23.3.19 upgrade()

```
void Tower::upgrade ( )
```

Upgrade the tower attack speed.

5.23.4 Member Data Documentation

5.23.4.1 attack_speed_

```
int Tower::attack_speed_ [protected]
```

5.23.4.2 font_

```
sf::Font Tower::font_ [private]
```

5.23.4.3 has_target_

```
bool Tower::has_target_ = false [protected]
```

5.23.4.4 last_fire_time_

```
sf::Time Tower::last_fire_time_ [private]
```

5.23.4.5 level_

```
int Tower::level_ = 1 [protected]
```

5.23.4.6 price_

```
int Tower::price_ [protected]
```

5.23.4.7 range_

```
float Tower::range_ [protected]
```

5.23.4.8 range_circle_

```
sf::CircleShape Tower::range_circle_ [protected]
```

5.23.4.9 selected_

```
bool Tower::selected_ [private]
```

5.23.4.10 sprite_

```
sf::Sprite Tower::sprite_ [protected]
```

5.23.4.11 target_

```
std::shared_ptr<Enemy> Tower::target_ [protected]
```

5.23.4.12 text_

```
sf::Text Tower::text_ [private]
```

5.23.4.13 texture_

```
sf::Texture Tower::texture_ [private]
```

5.23.4.14 type_

```
int Tower::type_ [private]
```

The documentation for this class was generated from the following files:

- src/[Tower.hpp](#)
- src/[Tower.cpp](#)

5.24 Wave Class Reference

Class that generates individual waves.

```
#include <Wave.hpp>
```

Public Member Functions

- [Wave](#) (int waveNum, const std::shared_ptr< [Node](#) > &start)
Construct a new [Wave](#) object.
- std::vector< std::shared_ptr< [Enemy](#) > > & [getEnemies](#) ()
Get the vector of Enemies.

Private Attributes

- std::vector< std::shared_ptr< [Enemy](#) > > [enemies_](#)

5.24.1 Detailed Description

Class that generates individual waves.

5.24.2 Constructor & Destructor Documentation

5.24.2.1 Wave()

```
Wave::Wave (  
    int waveNum,  
    const std::shared_ptr< Node > & start )
```

Construct a new [Wave](#) object.

Parameters

<i>waveNum</i>	wavenumber that affects wave creation
<i>start</i>	starting point for enemies

5.24.3 Member Function Documentation

5.24.3.1 getEnemies()

```
std::vector< std::shared_ptr< Enemy > > & Wave::getEnemies ( )
```

Get the vector of Enemies.

Returns

```
std::vector<std::shared_ptr<Enemy>>&
```

5.24.4 Member Data Documentation

5.24.4.1 enemies_

```
std::vector<std::shared_ptr<Enemy> > Wave::enemies_ [private]
```

The documentation for this class was generated from the following files:

- src/[Wave.hpp](#)
- src/[Wave.cpp](#)

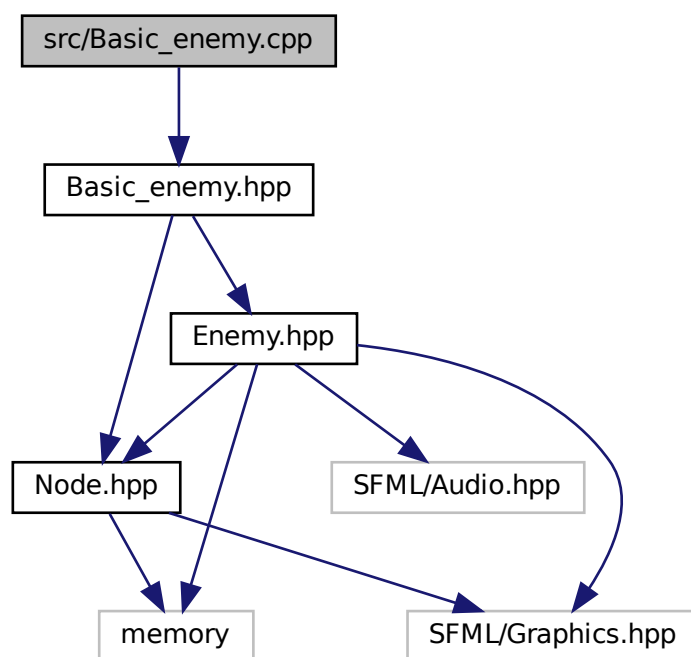
Chapter 6

File Documentation

6.1 src/Basic_enemy.cpp File Reference

```
#include "Basic_enemy.hpp"
```

Include dependency graph for Basic_enemy.cpp:

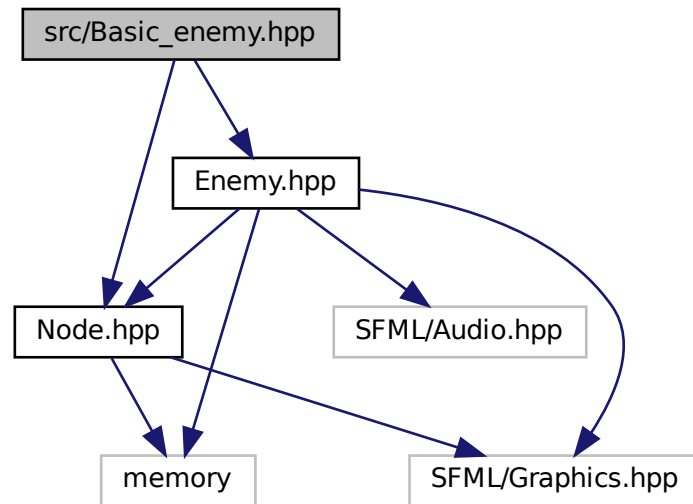


6.2 src/Basic_enemy.hpp File Reference

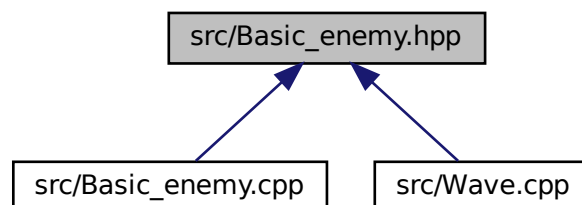
```
#include "Enemy.hpp"
```

```
#include "Node.hpp"
```

Include dependency graph for Basic_enemy.hpp:



This graph shows which files directly or indirectly include this file:



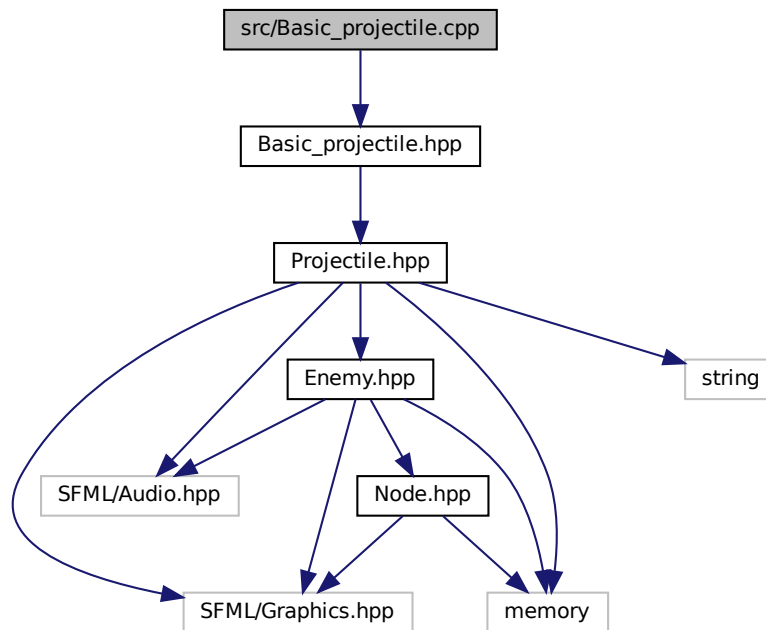
Classes

- class [BasicEnemy](#)
Child class for basic enemies.

6.3 src/Basic_projectile.cpp File Reference

```
#include "Basic_projectile.hpp"
```

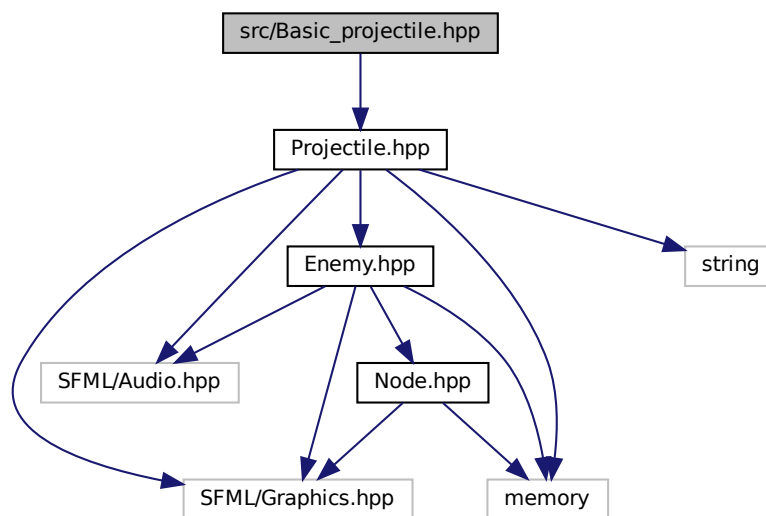
Include dependency graph for Basic_projectile.cpp:



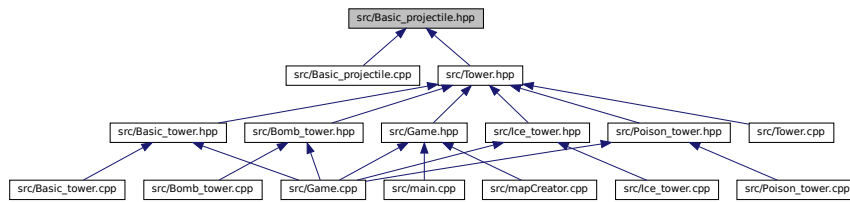
6.4 src/Basic_projectile.hpp File Reference

```
#include "Projectile.hpp"
```

Include dependency graph for Basic_projectile.hpp:



This graph shows which files directly or indirectly include this file:



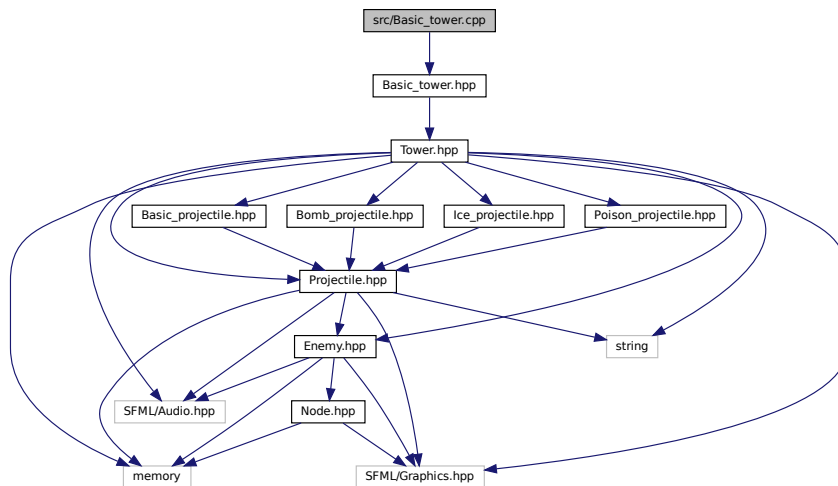
Classes

- class [basicProjectile](#)
Basic projectile for basic tower.

6.5 src/Basic_tower.cpp File Reference

```
#include "Basic_tower.hpp"
```

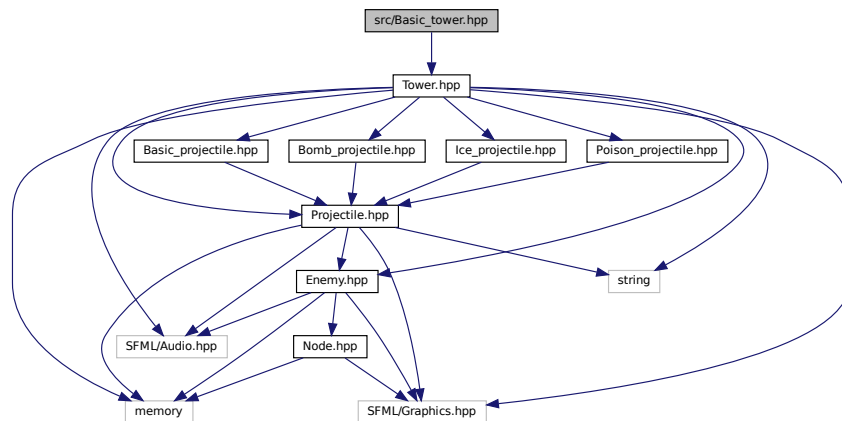
Include dependency graph for Basic_tower.cpp:



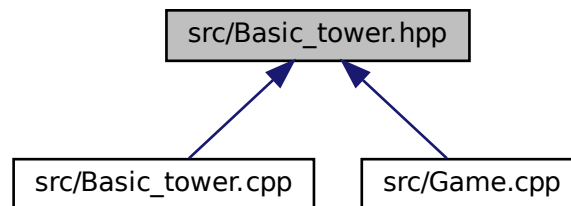
6.6 src/Basic_tower.hpp File Reference

```
#include "Tower.hpp"
```

Include dependency graph for Basic_tower.hpp:



This graph shows which files directly or indirectly include this file:



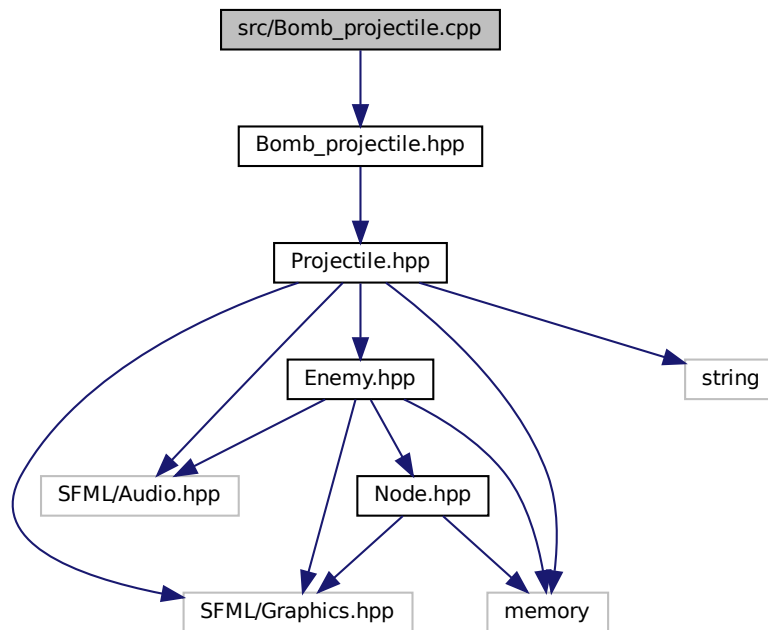
Classes

- class [basicTower](#)
Basic tower class.

6.7 src/Bomb_projectile.cpp File Reference

```
#include "Bomb_projectile.hpp"
```

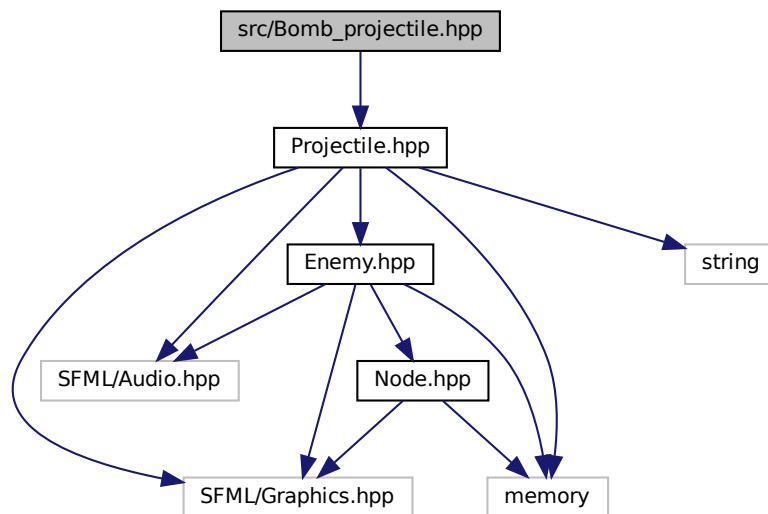
Include dependency graph for Bomb_projectile.cpp:



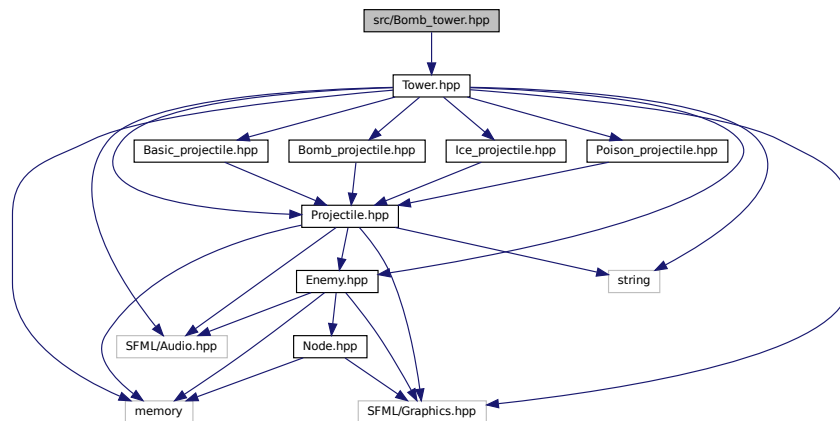
6.8 src/Bomb_projectile.hpp File Reference

```
#include "Projectile.hpp"
```

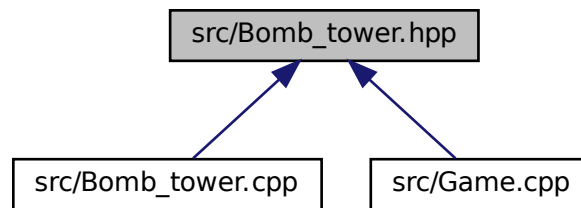
Include dependency graph for Bomb_projectile.hpp:



Include dependency graph for Bomb_tower.hpp:



This graph shows which files directly or indirectly include this file:



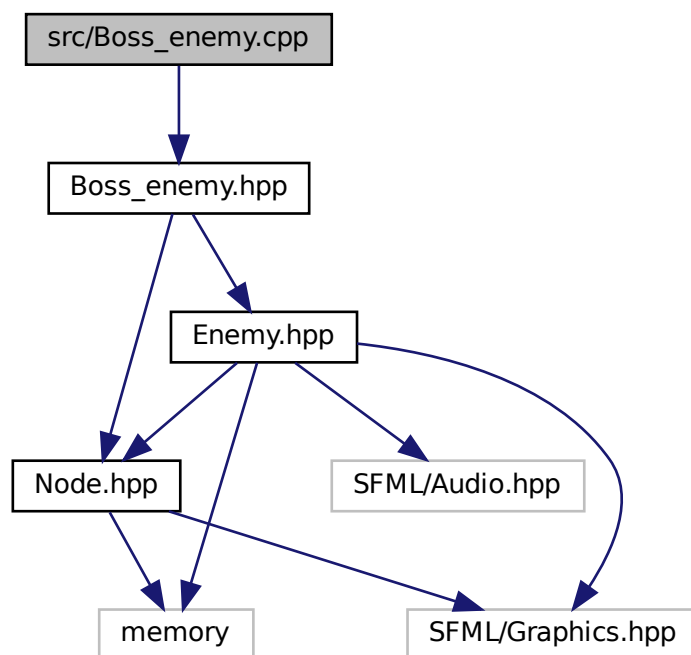
Classes

- class [BombTower](#)
Bomb tower class.

6.11 src/Boss_enemy.cpp File Reference

```
#include "Boss_enemy.hpp"
```

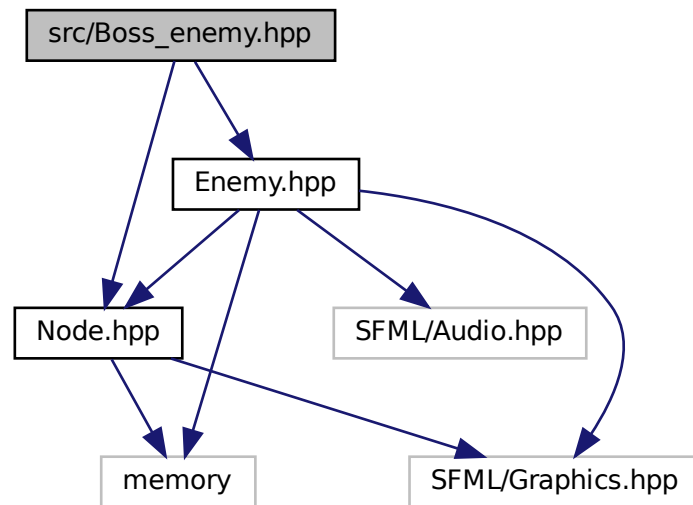

Include dependency graph for Boss_enemy.cpp:



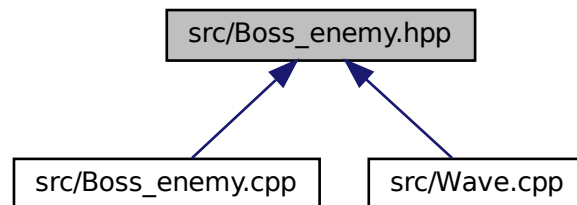
6.12 src/Boss_enemy.hpp File Reference

```
#include "Enemy.hpp"  
#include "Node.hpp"
```

Include dependency graph for Boss_enemy.hpp:



This graph shows which files directly or indirectly include this file:



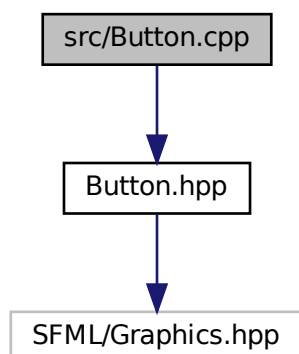
Classes

- class [BossEnemy](#)
Child class for boss enemies.

6.13 src/Button.cpp File Reference

```
#include "Button.hpp"
```

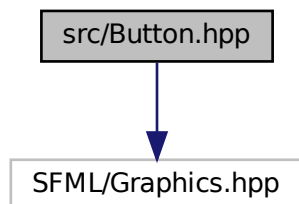
Include dependency graph for Button.cpp:



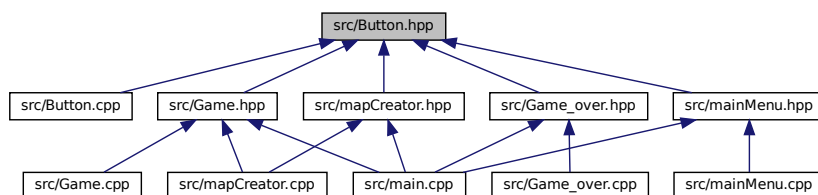
6.14 src/Button.hpp File Reference

```
#include <SFML/Graphics.hpp>
```

Include dependency graph for Button.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [Button](#)

Class for implementing buttons using SFML.

6.14.1 Detailed Description

Author

David Enberg (david.enberg@aalto.fi)

Version

0.1

Date

2022-12-06

Copyright

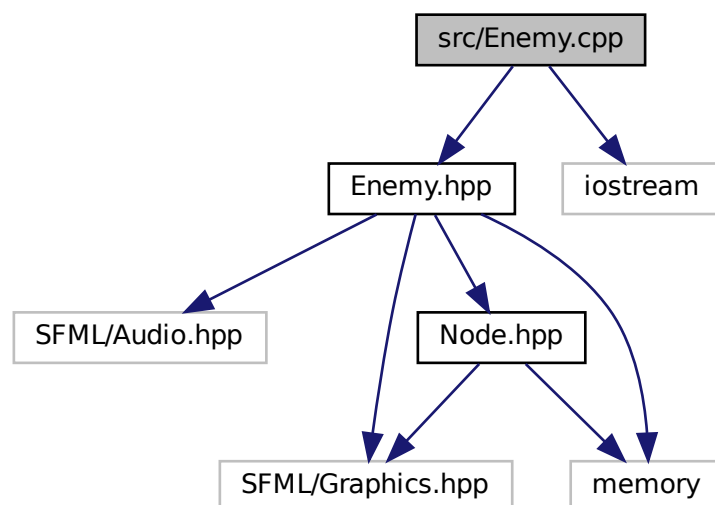
Copyright (c) 2022

6.15 src/Enemy.cpp File Reference

```
#include "Enemy.hpp"
```

```
#include <iostream>
```

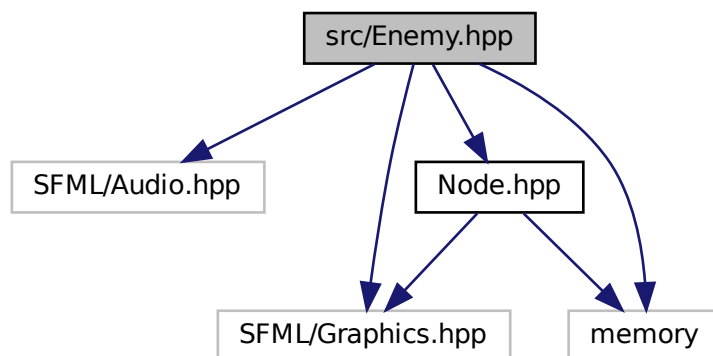
Include dependency graph for Enemy.cpp:



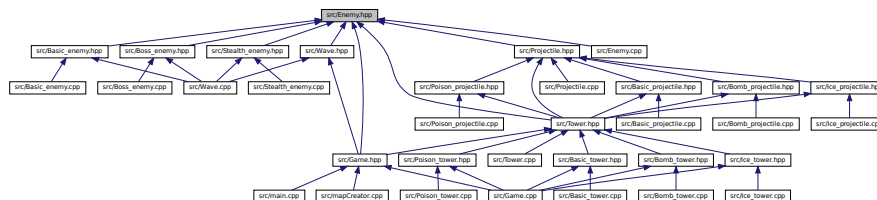
6.16 src/Enemy.hpp File Reference

Parent class for all enemies.

```
#include <SFML/Audio.hpp>
#include <SFML/Graphics.hpp>
#include <memory>
#include "Node.hpp"
Include dependency graph for Enemy.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [Enemy](#)
Enemy parent class.

6.16.1 Detailed Description

Parent class for all enemies.

Author

David Enberg david.enberg@aalto.fi

Version

0.1

Date

2022-11-09

Copyright

Copyright (c) 2022

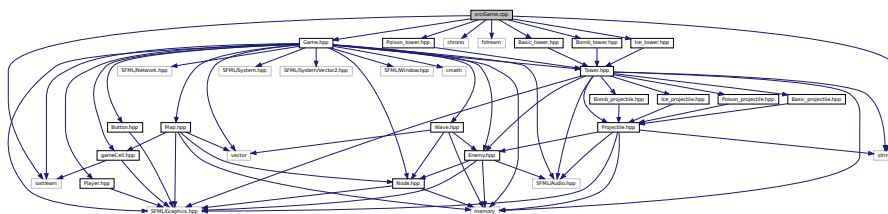
6.17 src/Game.cpp File Reference

```

#include "Game.hpp"
#include <chrono>
#include <fstream>
#include <iostream>
#include <string>
#include "Basic_tower.hpp"
#include "Bomb_tower.hpp"
#include "Ice_tower.hpp"
#include "Poison_tower.hpp"

```

Include dependency graph for Game.cpp:

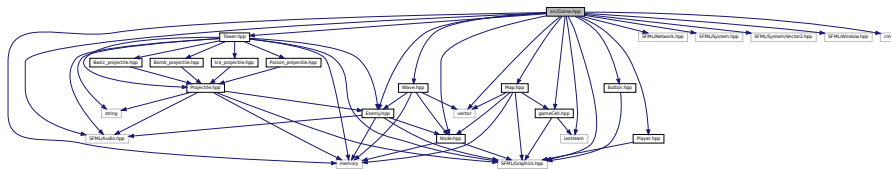
**6.18 src/Game.hpp File Reference**

```

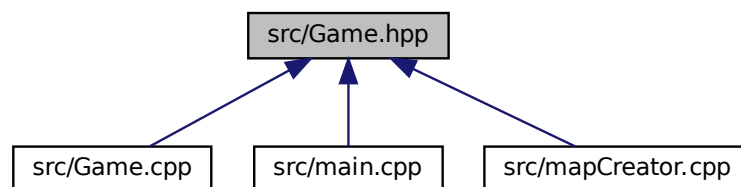
#include <SFML/Audio.hpp>
#include <SFML/Graphics.hpp>
#include <SFML/Network.hpp>
#include <SFML/System.hpp>
#include <SFML/System/Vector2.hpp>
#include <SFML/Window.hpp>
#include <cmath>
#include <iostream>
#include <memory>
#include <vector>
#include "Button.hpp"
#include "Enemy.hpp"
#include "Map.hpp"
#include "Node.hpp"
#include "Player.hpp"

```

```
#include "Tower.hpp"
#include "Wave.hpp"
#include "gameCell.hpp"
Include dependency graph for Game.hpp:
```



This graph shows which files directly or indirectly include this file:



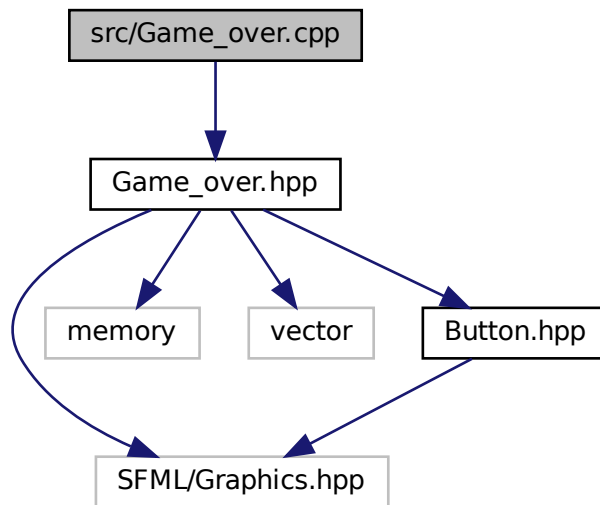
Classes

- class [Game](#)

6.19 src/Game_over.cpp File Reference

```
#include "Game_over.hpp"
```

Include dependency graph for Game_over.cpp:

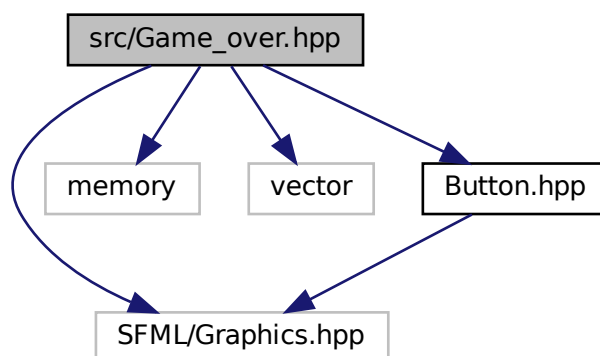


6.20 src/Game_over.hpp File Reference

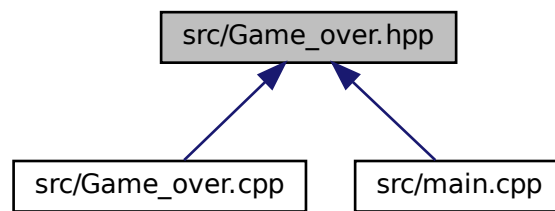
Handles state after game finishes execution.

```
#include <SFML/Graphics.hpp>
#include <memory>
#include <vector>
#include "Button.hpp"
```

Include dependency graph for Game_over.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [GameOver](#)
Handles state after game finishes execution.

6.20.1 Detailed Description

Handles state after game finishes execution.

Author

David Enberg (david.enberg@aalto.fi)

Version

0.1

Date

2022-12-06

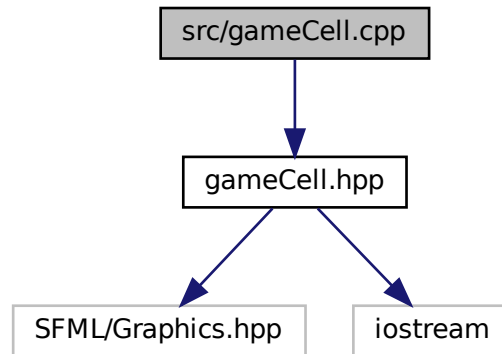
Copyright

Copyright (c) 2022

6.21 src/gameCell.cpp File Reference

```
#include "gameCell.hpp"
```

Include dependency graph for gameCell.cpp:



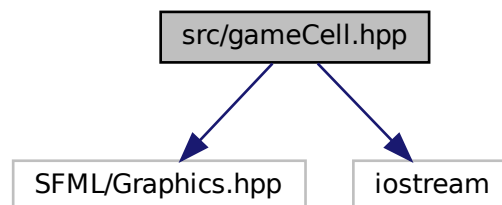
6.22 src/gameCell.hpp File Reference

Splitting the map into cells that are easier to handle.

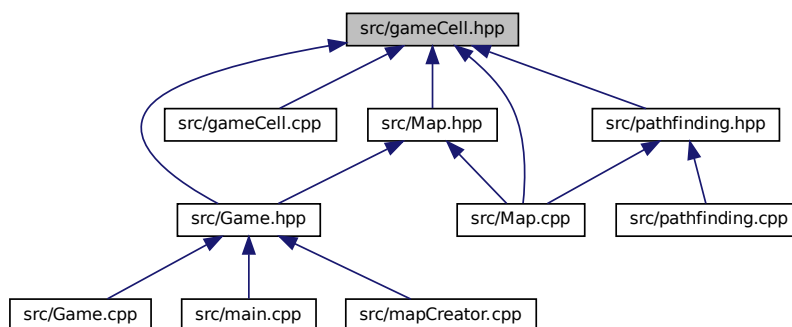
```
#include <SFML/Graphics.hpp>
```

```
#include <iostream>
```

Include dependency graph for gameCell.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [GameCell](#)

6.22.1 Detailed Description

Splitting the map into cells that are easier to handle.

Author

Axel Hedman, axel.hedman@aalto.fi

Version

0.1

Date

2022-11-13

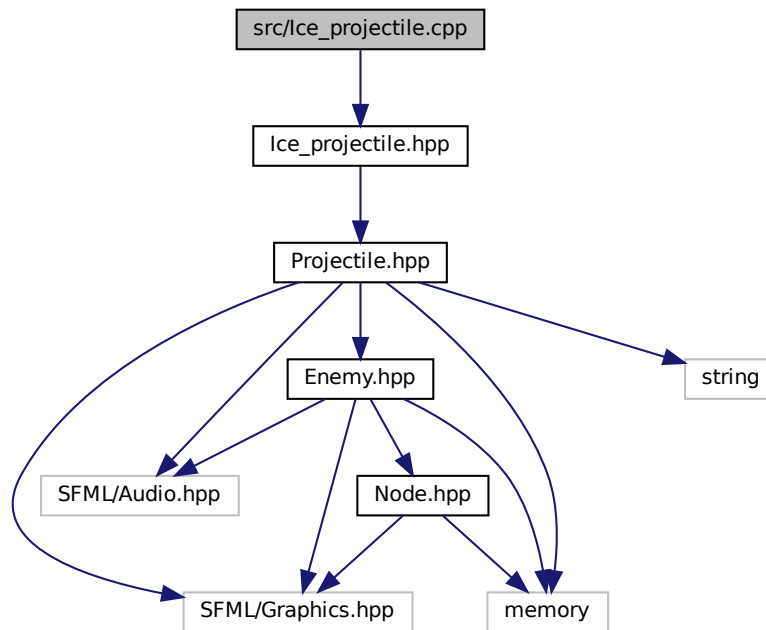
Copyright

Copyright (c) 2022

6.23 src/Ice_projectile.cpp File Reference

```
#include "Ice_projectile.hpp"
```

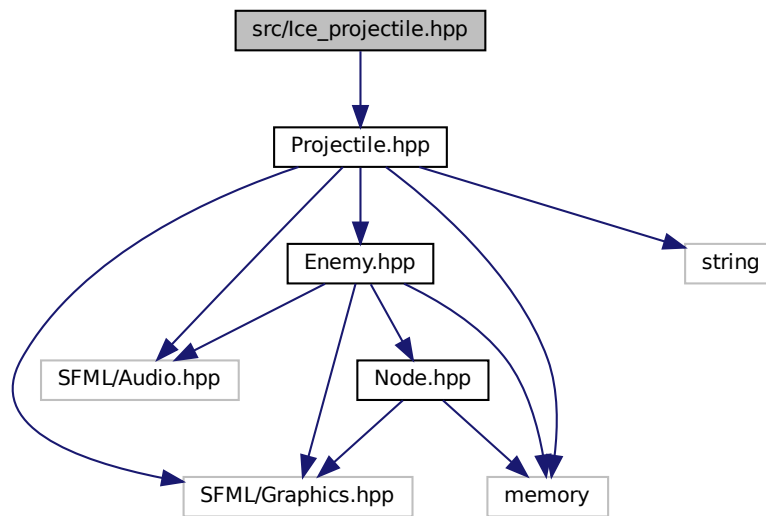
Include dependency graph for Ice_projectile.cpp:



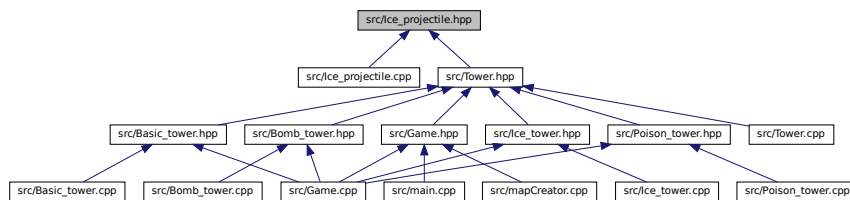
6.24 src/Ice_projectile.hpp File Reference

```
#include "Projectile.hpp"
```

Include dependency graph for Ice_projectile.hpp:



This graph shows which files directly or indirectly include this file:



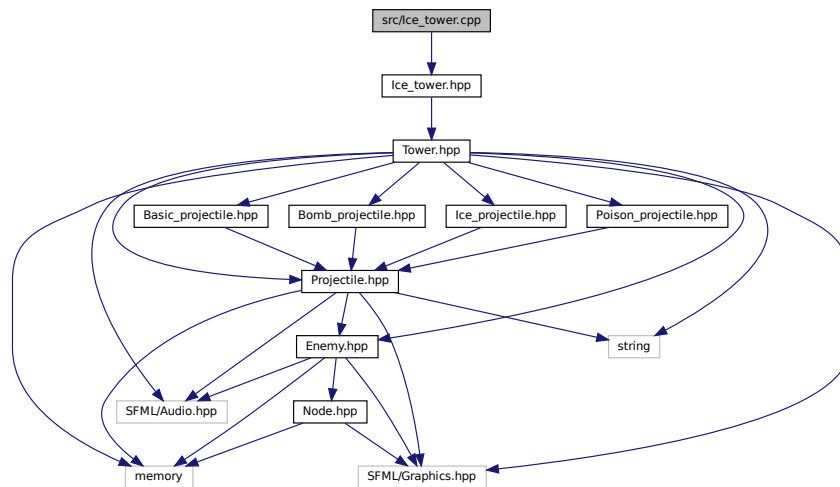
Classes

- class `IceProjectile`
Ice projectile for ice tower.

6.25 src/Ice_tower.cpp File Reference

```
#include "Ice_tower.hpp"
```

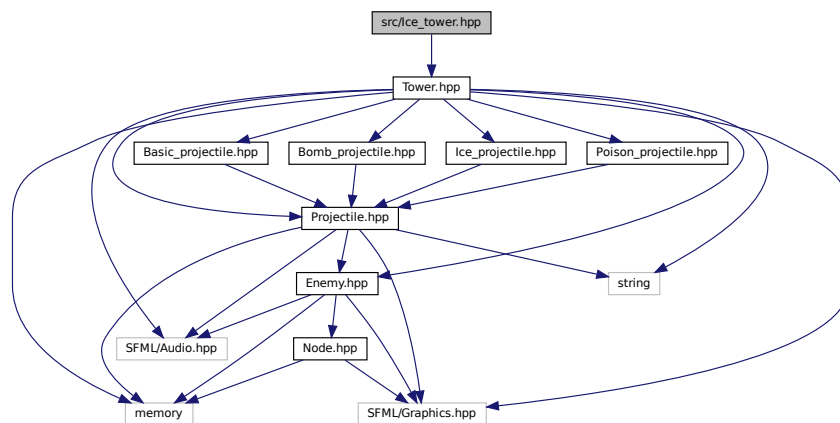
Include dependency graph for Ice_tower.cpp:



6.26 src/Ice_tower.hpp File Reference

```
#include "Tower.hpp"
```

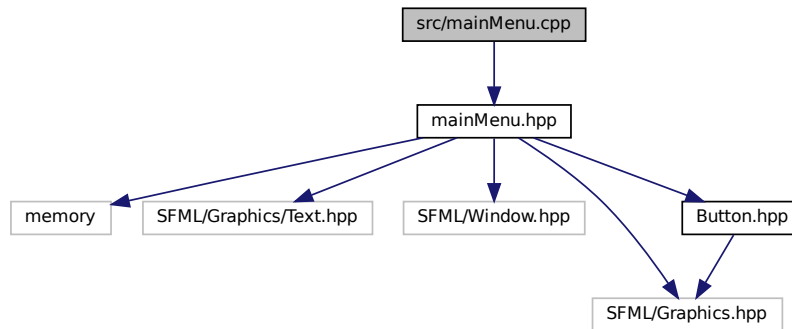
Include dependency graph for Ice_tower.hpp:



6.28 src/mainMenu.cpp File Reference

```
#include "mainMenu.hpp"
```

Include dependency graph for mainMenu.cpp:

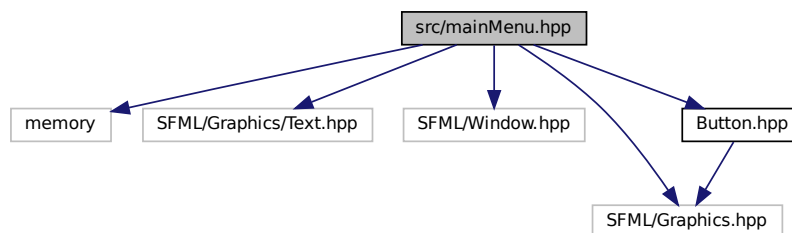


6.29 src/mainMenu.hpp File Reference

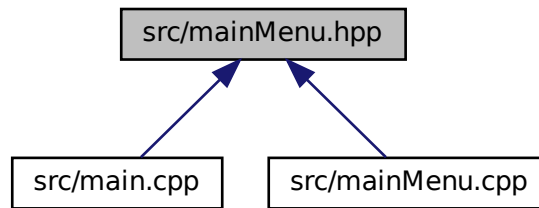
Main Menu Implementation.

```
#include <memory>
#include <SFML/Graphics/Text.hpp>
#include <SFML/Window.hpp>
#include <SFML/Graphics.hpp>
#include "Button.hpp"
```

Include dependency graph for mainMenu.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class `mainMenu`

6.29.1 Detailed Description

Main Menu Implementation.

Author

Axel Hedman, axel.hedman@aalto.fi

Version

0.1

Date

2022-12-2

Copyright

Copyright (c) 2022

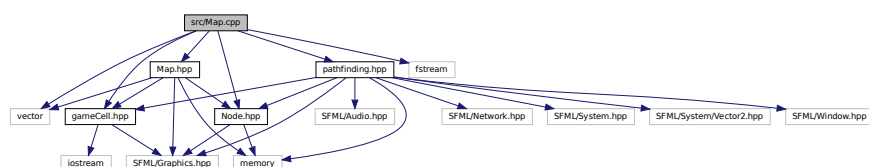
6.30 src/Map.cpp File Reference

```

#include "Map.hpp"
#include <fstream>
#include <vector>
#include "Node.hpp"
#include "gameCell.hpp"
#include "pathfinding.hpp"

```

Include dependency graph for Map.cpp:

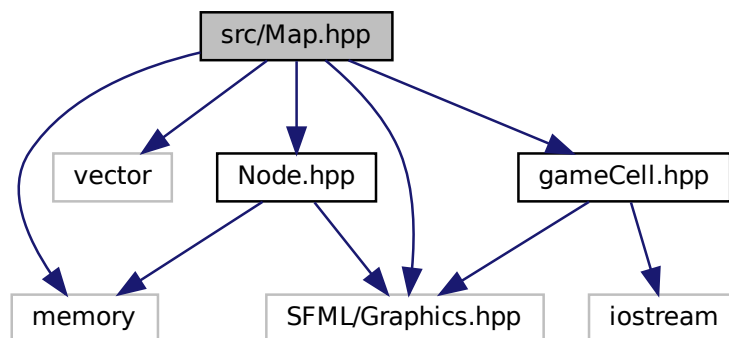


6.31 src/Map.hpp File Reference

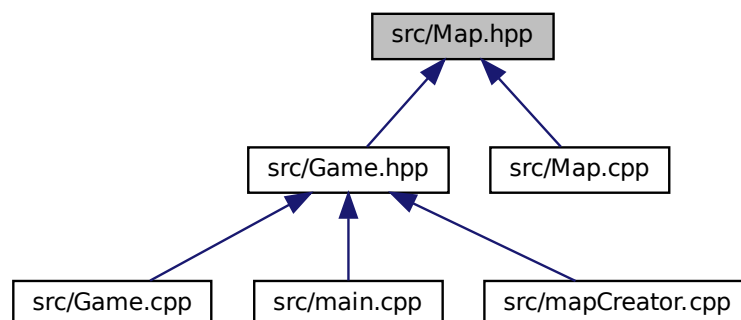
[Map](#) Class Implementation.

```
#include <SFML/Graphics.hpp>
#include <vector>
#include <memory>
#include "gameCell.hpp"
#include "Node.hpp"
```

Include dependency graph for Map.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [Map](#)

6.31.1 Detailed Description

[Map](#) Class Implementation.

Author

Axel Hedman, axel.hedman@aalto.fi

Version

0.1

Date

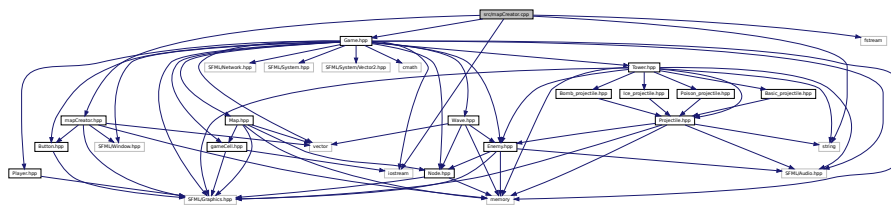
2022-11-22

Copyright

Copyright (c) 2022

6.32 src/mapCreator.cpp File Reference

```
#include <iostream>
#include <fstream>
#include <string>
#include "mapCreator.hpp"
#include "Game.hpp"
Include dependency graph for mapCreator.cpp:
```



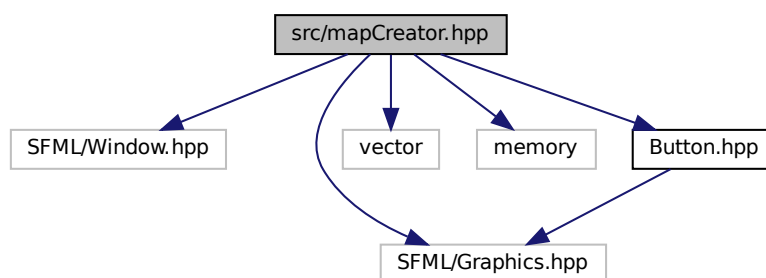
6.33 src/mapCreator.hpp File Reference

[Map](#) Creator Implementation.

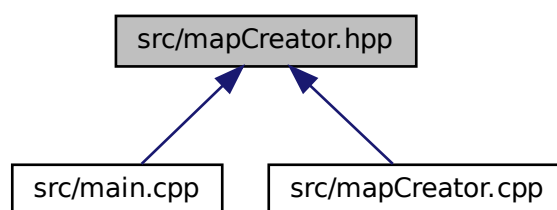
```
#include <SFML/Window.hpp>
#include <SFML/Graphics.hpp>
#include <vector>
#include <memory>
```

```
#include "Button.hpp"
```

Include dependency graph for mapCreator.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [mapCreator](#)

6.33.1 Detailed Description

[Map](#) Creator Implementation.

Author

Axel Hedman, axel.hedman@aalto.fi

Version

0.1

Date

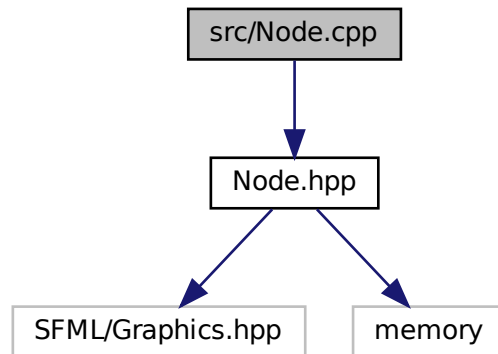
2022-11-27

Copyright

Copyright (c) 2022

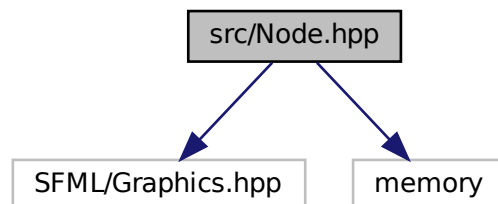
6.34 src/Node.cpp File Reference

```
#include "Node.hpp"
Include dependency graph for Node.cpp:
```



6.35 src/Node.hpp File Reference

```
#include <SFML/Graphics.hpp>
#include <memory>
Include dependency graph for Node.hpp:
```



This graph shows which files directly or indirectly include this file:



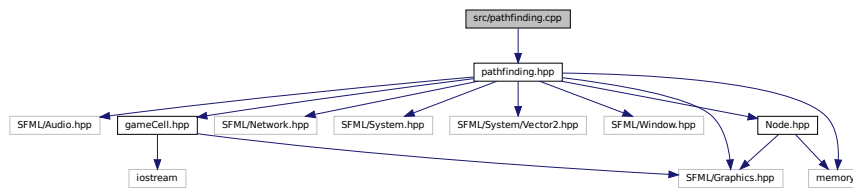
Classes

- class [Node](#)

6.36 src/pathfinding.cpp File Reference

```
#include "pathfinding.hpp"
```

Include dependency graph for pathfinding.cpp:



Functions

- bool [isValid](#) (int x, int y)
Check if coordinate is inside map.
- void [pathSearch](#) (const std::vector< std::vector< std::shared_ptr< [GameCell](#) >>> &grid, std::vector< std::shared_ptr< [Node](#) >> &nodes, const sf::Vector2f &start)
Search for the path.

6.36.1 Function Documentation

6.36.1.1 isValid()

```
bool isValid (
    int x,
    int y )
```

Check if coordinate is inside map.

Parameters

<i>x</i>	
<i>y</i>	

Returns

true if coordinate is inside file
false if coordinate is not inside file

6.36.1.2 pathSearch()

```
void pathSearch (
    const std::vector< std::vector< std::shared_ptr< GameCell >>> & grid,
    std::vector< std::shared_ptr< Node >> & nodes,
    const sf::Vector2f & start )
```

Search for the path.

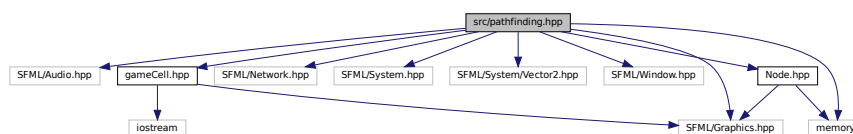
Parameters

<i>grid</i>	Map grid
<i>nodes</i>	Vector to where path will be added
<i>start</i>	Coordinate of start

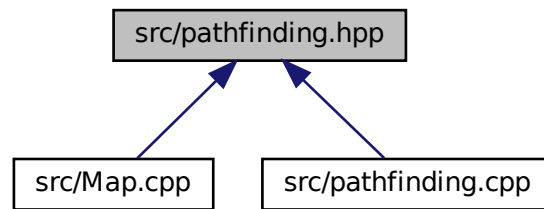
6.37 src/pathfinding.hpp File Reference

```
#include <SFML/Audio.hpp>
#include <SFML/Graphics.hpp>
#include <SFML/Network.hpp>
#include <SFML/System.hpp>
#include <SFML/System/Vector2.hpp>
#include <SFML/Window.hpp>
#include "gameCell.hpp"
#include "Node.hpp"
#include <memory>
```

Include dependency graph for pathfinding.hpp:



This graph shows which files directly or indirectly include this file:



Functions

- bool `isValid` (int x, int y)
Check if coordinate is inside map.
- void `pathSearch` (const std::vector< std::vector< std::shared_ptr< `GameCell` >>> &grid, std::vector< std::shared_ptr< `Node` >> &nodes, const sf::Vector2f &start)
Search for the path.

6.37.1 Function Documentation

6.37.1.1 isValid()

```
bool isValid (
    int x,
    int y )
```

Check if coordinate is inside map.

Parameters

<i>x</i>	
<i>y</i>	

Returns

true if coordinate is inside file
false if coordinate is not inside file

6.37.1.2 pathSearch()

```
void pathSearch (
    const std::vector< std::vector< std::shared_ptr< GameCell >>> & grid,
    std::vector< std::shared_ptr< Node >> & nodes,
    const sf::Vector2f & start )
```

Search for the path.

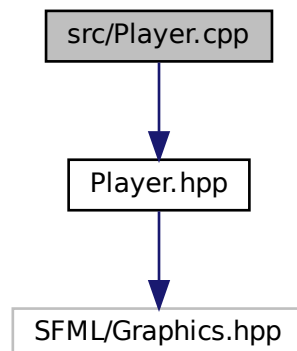
Parameters

<i>grid</i>	Map grid
<i>nodes</i>	Vector to where path will be added
<i>start</i>	Coordinate of start

6.38 src/Player.cpp File Reference

```
#include "Player.hpp"
```

Include dependency graph for Player.cpp:

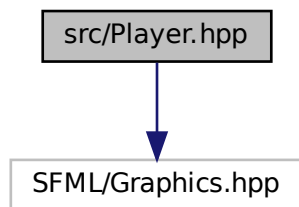


6.39 src/Player.hpp File Reference

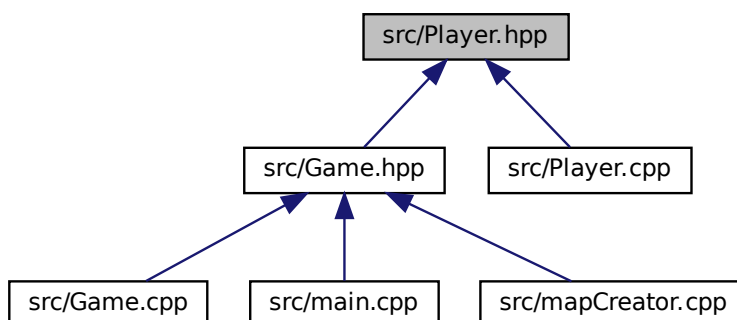
Class for handling status of player.

```
#include <SFML/Graphics.hpp>
```

Include dependency graph for Player.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [Player](#)

Class for handling gamestate from player perspective.

6.39.1 Detailed Description

Class for handling status of player.

Author

David Enberg (david.enberg@aalto.fi)

Version

0.1

Date

2022-12-06

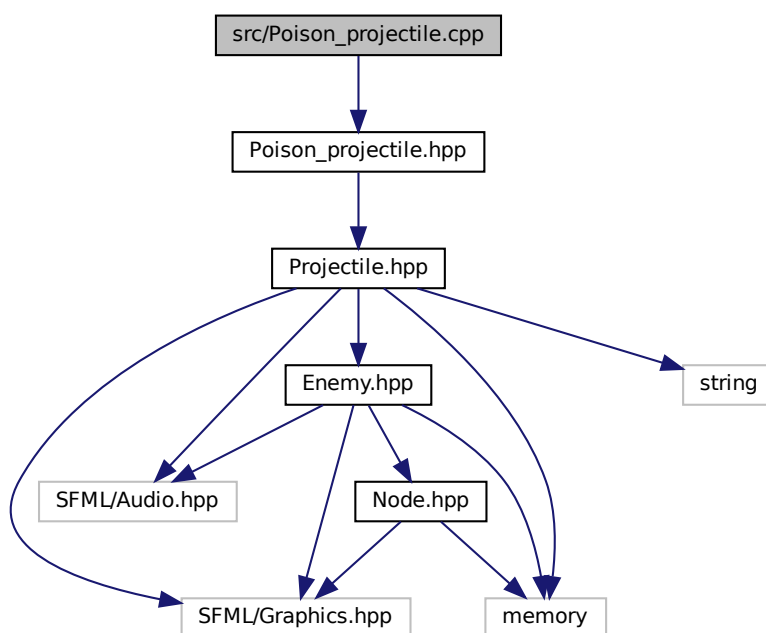
Copyright

Copyright (c) 2022

6.40 src/Poison_projectile.cpp File Reference

```
#include "Poison_projectile.hpp"
```

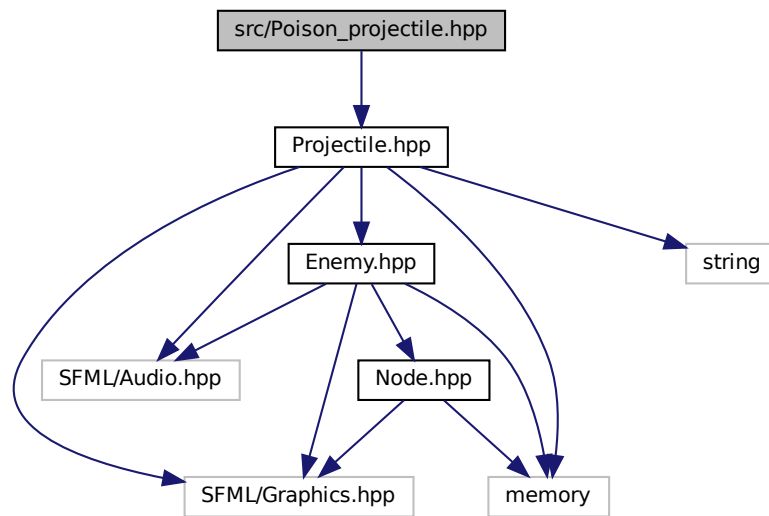
Include dependency graph for Poison_projectile.cpp:



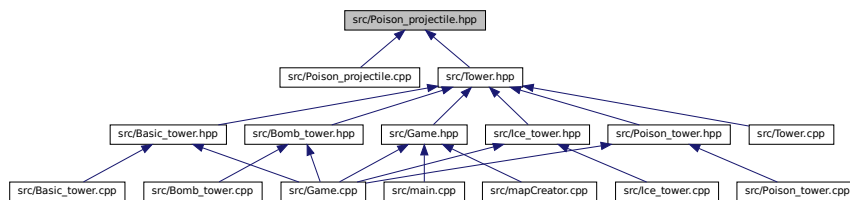
6.41 src/Poison_projectile.hpp File Reference

```
#include "Projectile.hpp"
```

Include dependency graph for Poison_projectile.hpp:



This graph shows which files directly or indirectly include this file:



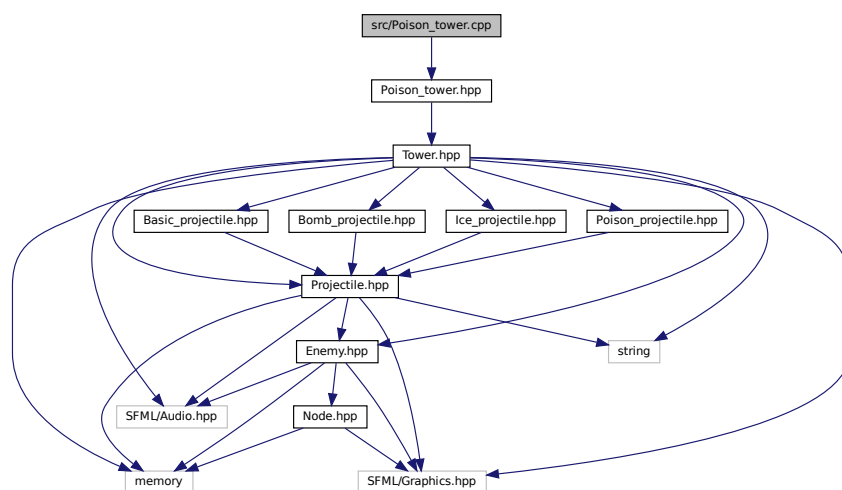
Classes

- class [PoisonProjectile](#)
Poison projectile for poison tower.

6.42 src/Poison_tower.cpp File Reference

```
#include "Poison_tower.hpp"
```

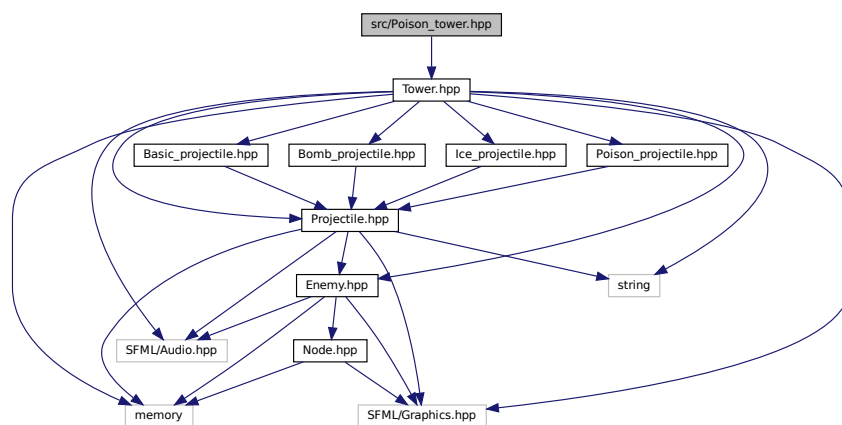
Include dependency graph for Poison_tower.cpp:



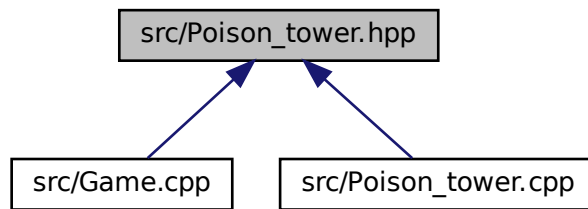
6.43 src/Poison_tower.hpp File Reference

```
#include "Tower.hpp"
```

Include dependency graph for Poison_tower.hpp:



This graph shows which files directly or indirectly include this file:



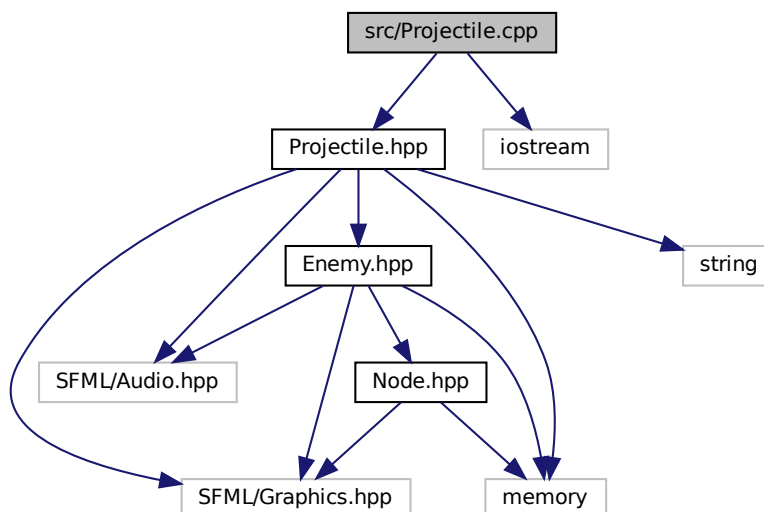
Classes

- class [PoisonTower](#)
Poison tower class.

6.44 src/Projectile.cpp File Reference

```
#include "Projectile.hpp"
#include <iostream>
```

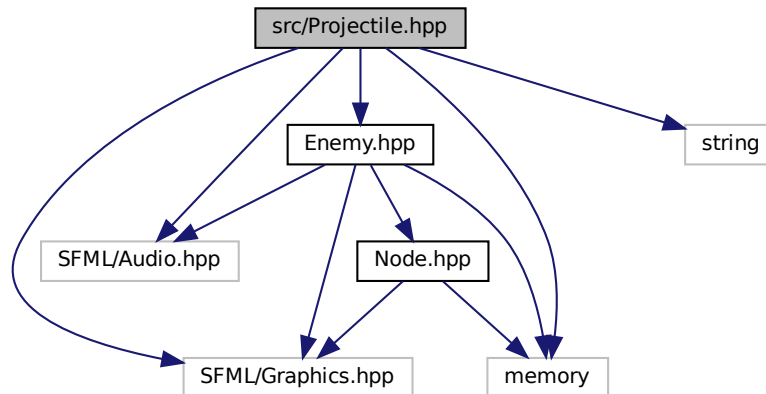
Include dependency graph for Projectile.cpp:



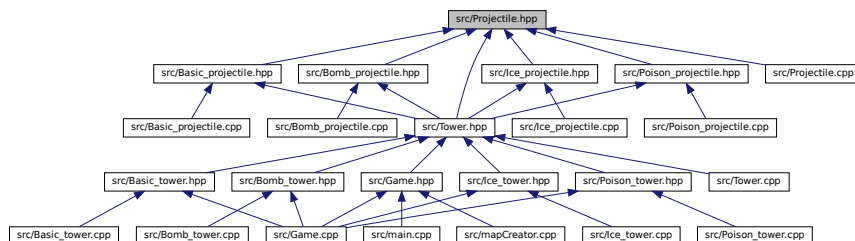
6.45 src/Projectile.hpp File Reference

```
#include <SFML/Audio.hpp>
#include <SFML/Graphics.hpp>
#include <memory>
#include <string>
#include "Enemy.hpp"
```

Include dependency graph for Projectile.hpp:



This graph shows which files directly or indirectly include this file:



Classes

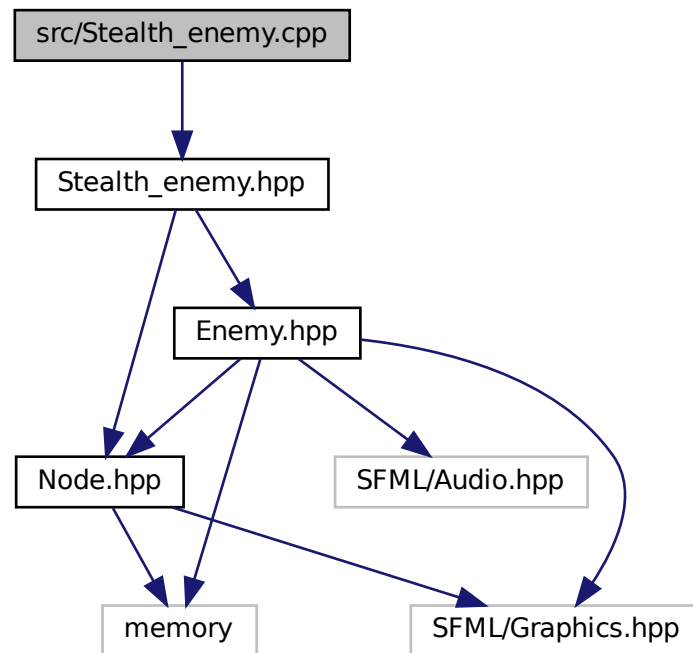
- class [Projectile](#)
Parent class for projectiles.

6.46 src/readme.md File Reference

6.47 src/Stealth_enemy.cpp File Reference

```
#include "Stealth_enemy.hpp"
```

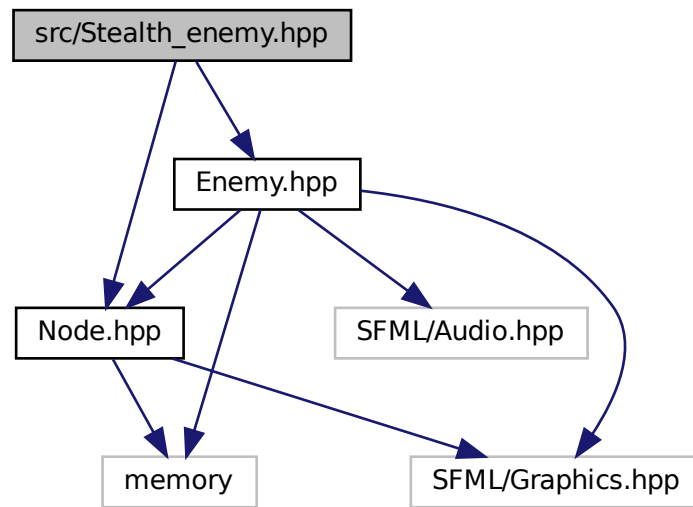
Include dependency graph for `Stealth_enemy.cpp`:



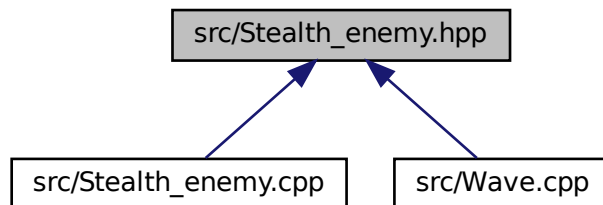
6.48 `src/Stealth_enemy.hpp` File Reference

```
#include "Enemy.hpp"  
#include "Node.hpp"
```


Include dependency graph for Stealth_enemy.hpp:



This graph shows which files directly or indirectly include this file:



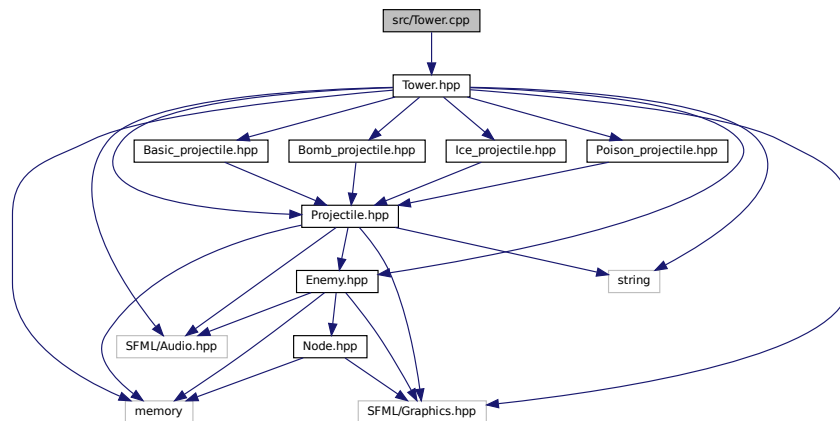
Classes

- class [StealthEnemy](#)
Child class for stealth enemies.

6.49 src/Tower.cpp File Reference

```
#include "Tower.hpp"
```

Include dependency graph for Tower.cpp:



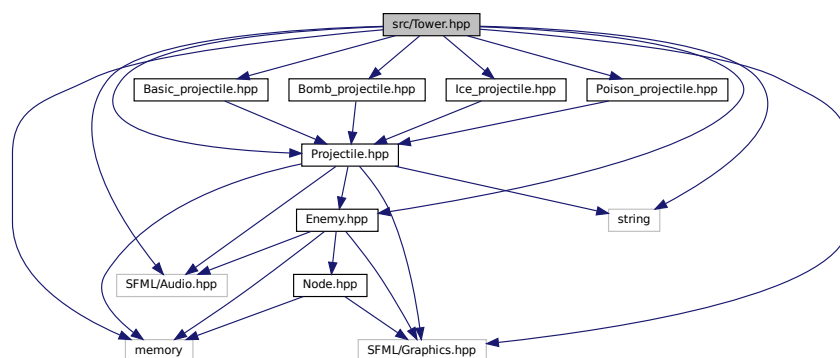
6.50 src/Tower.hpp File Reference

```

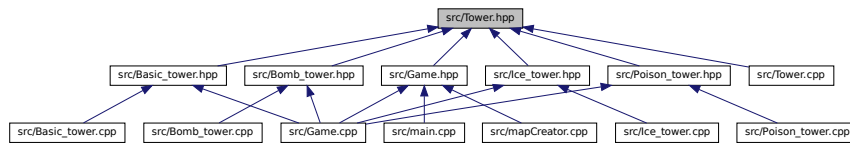
#include <SFML/Audio.hpp>
#include <SFML/Graphics.hpp>
#include <memory>
#include <string>
#include "Basic_projectile.hpp"
#include "Bomb_projectile.hpp"
#include "Enemy.hpp"
#include "Ice_projectile.hpp"
#include "Poison_projectile.hpp"
#include "Projectile.hpp"

```

Include dependency graph for Tower.hpp:



This graph shows which files directly or indirectly include this file:

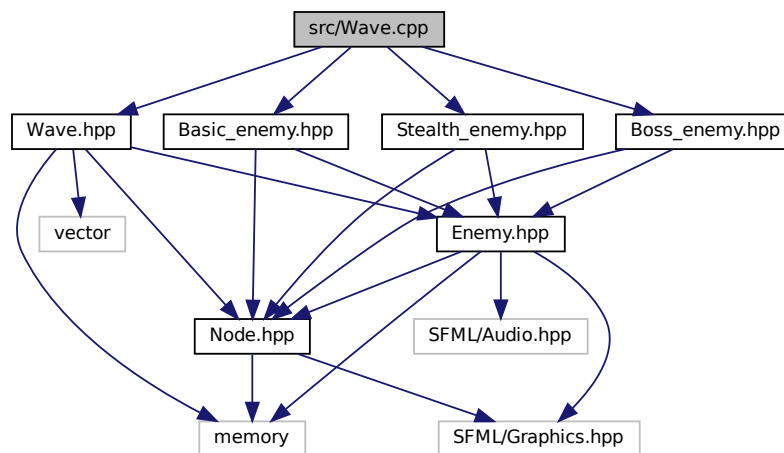


Classes

- class [Tower](#)
Parent class for different towers.

6.51 src/Wave.cpp File Reference

```
#include "Wave.hpp"
#include "Basic_enemy.hpp"
#include "Stealth_enemy.hpp"
#include "Boss_enemy.hpp"
Include dependency graph for Wave.cpp:
```

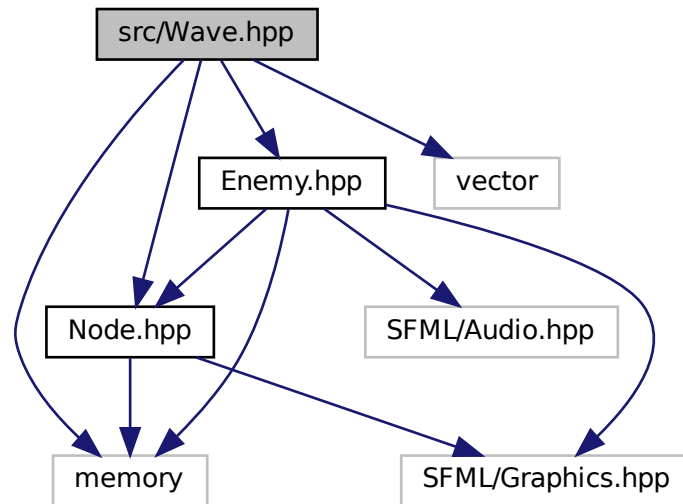


6.52 src/Wave.hpp File Reference

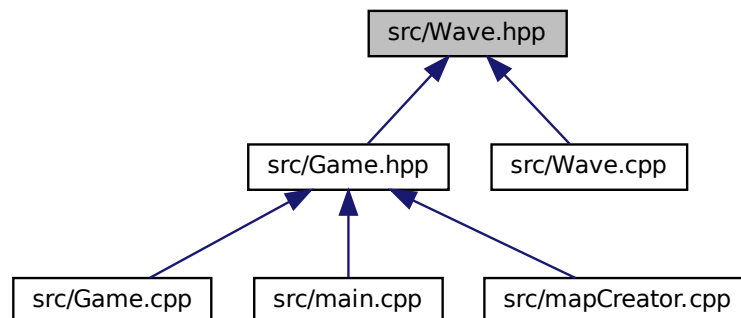
```
#include <memory>
#include <vector>
#include "Enemy.hpp"
```

```
#include "Node.hpp"
```

Include dependency graph for Wave.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [Wave](#)

Class that generates individual waves.

6.52.1 Detailed Description

Author

David Enberg (david.enberg@aalto.fi)

Version

0.1

Date

2022-12-06

Copyright

Copyright (c) 2022

Index

- ~BombTower
 - BombTower, [17](#)
- ~Enemy
 - Enemy, [28](#)
- ~Game
 - Game, [38](#)
- ~GameCell
 - GameCell, [45](#)
- ~IceTower
 - IceTower, [52](#)
- ~Map
 - Map, [59](#)
- ~PoisonTower
 - PoisonTower, [72](#)
- ~Projectile
 - Projectile, [75](#)
- ~Tower
 - Tower, [81](#)
- ~basicTower
 - basicTower, [13](#)
- ~mainMenu
 - mainMenu, [54](#)
- ~mapCreator
 - mapCreator, [61](#)
- alive_
 - Enemy, [34](#)
- atCheckpoint
 - Enemy, [28](#)
- attack_speed_
 - Tower, [86](#)
- BasicEnemy, [9](#)
 - BasicEnemy, [10](#)
- basicProjectile, [10](#)
 - basicProjectile, [11](#)
- basicTower, [12](#)
 - ~basicTower, [13](#)
 - basicTower, [13](#)
 - fire, [13](#)
 - getProjectiles, [14](#)
 - projectiles_, [14](#)
- BombProjectile, [15](#)
 - BombProjectile, [16](#)
- BombTower, [16](#)
 - ~BombTower, [17](#)
 - BombTower, [17](#)
 - fire, [18](#)
 - getProjectiles, [18](#)
 - projectiles_, [18](#)
- BossEnemy, [19](#)
 - BossEnemy, [20](#)
- Button, [20](#)
 - Button, [21](#)
 - clicked_, [24](#)
 - font_, [24](#)
 - getSprite, [22](#)
 - getText, [22](#)
 - getType, [22](#)
 - is_clicked_, [25](#)
 - is_Finnish, [25](#)
 - is_Start, [25](#)
 - isClicked, [22](#)
 - isFinnish, [22](#)
 - isStart, [23](#)
 - notClicked_, [25](#)
 - setClicked, [23](#)
 - setFinnish, [23](#)
 - setNotClicked, [23](#)
 - setNotFinnish, [23](#)
 - setNotStart, [24](#)
 - setSprite, [24](#)
 - setStart, [24](#)
 - sprite_, [25](#)
 - text_, [25](#)
 - texture_, [25](#)
 - type_, [25](#)
- button_background_
 - Game, [41](#)
- buttons_
 - Game, [42](#)
 - GameOver, [48](#)
 - mainMenu, [56](#)
 - mapCreator, [62](#)
- checkButtons
 - Game, [38](#)
 - GameOver, [47](#)
 - mapCreator, [61](#)
- checkFreeze
 - Enemy, [28](#)
- checkPoisonDamage
 - Enemy, [29](#)
- checkPositions
 - Game, [39](#)
- checkTowers
 - Game, [39](#)
- clicked_
 - Button, [24](#)
- clock

- Game, 42
- coordinate_
 - Node, 65
- damage_
 - Projectile, 77
- death_sound_
 - Enemy, 34
- death_sound_buffer_
 - Enemy, 34
- delta_x
 - Enemy, 34
- delta_y
 - Enemy, 34
- enemies_
 - Game, 42
 - Wave, 89
- Enemy, 26
 - ~Enemy, 28
 - alive_, 34
 - atCheckpoint, 28
 - checkFreeze, 28
 - checkPoisonDamage, 29
 - death_sound_, 34
 - death_sound_buffer_, 34
 - delta_x, 34
 - delta_y, 34
 - Enemy, 28
 - Freeze, 29
 - freeze_timer_, 34
 - frozen_, 35
 - getHitbox, 29
 - getHitpoints, 29
 - getPoisoned, 29
 - getSpeed, 30
 - getSprite, 30
 - getSteps, 30
 - getType, 30
 - has_payed_, 35
 - hitbox_, 35
 - hitpoints_, 35
 - isalive, 31
 - isFrozen, 31
 - kill, 31
 - move, 31
 - next_point_, 35
 - poison_timer_, 35
 - poisoned_, 35
 - setCheckpoint, 32
 - setHitpoints, 32
 - shouldPay, 32
 - speed_, 35
 - sprite_, 36
 - steps_taken_, 36
 - takeDamage, 32
 - texture_, 36
 - type_, 36
 - unFreeze, 34
 - x_goal_, 36
 - y_goal_, 36
- enemy_
 - Projectile, 77
- event_
 - Game, 42
 - GameOver, 49
 - mainMenu, 56
 - mapCreator, 62
- exitButton_
 - mainMenu, 56
- exitButtonPressed_
 - mainMenu, 56
- exitButtonSelected_
 - mainMenu, 56
- file_path_
 - Map, 59
- fire
 - basicTower, 13
 - BombTower, 18
 - IceTower, 53
 - PoisonTower, 73
 - Tower, 81
- font_
 - Button, 24
 - GameOver, 49
 - mainMenu, 56
 - Player, 69
 - Tower, 86
- Freeze
 - Enemy, 29
- freeze_timer_
 - Enemy, 34
- frozen_
 - Enemy, 35
- Game, 37
 - ~Game, 38
 - button_background_, 41
 - buttons_, 42
 - checkButtons, 38
 - checkPositions, 39
 - checkTowers, 39
 - clock, 42
 - enemies_, 42
 - event_, 42
 - Game, 38
 - intersects, 39
 - map_, 42
 - moveTower, 40
 - nodes_, 42
 - paused_, 42
 - placing_new_tower, 42
 - player_, 43
 - pollEv, 40
 - render, 40
 - running, 40
 - showErrorMessage, 40

- showInfo, 41
- state_, 43
- tiles_, 43
- towers_, 43
- update, 41
- validPlacement, 41
- wave_number_, 43
- window_, 43
- GameCell, 44
 - ~GameCell, 45
 - GameCell, 44
 - getSprite, 45
 - isExit_, 45
 - isPassable_, 45
 - isStart_, 46
 - pos_, 46
 - setUpSprite, 45
 - sprite_, 46
 - texture_, 46
- GameOver, 46
 - buttons_, 48
 - checkButtons, 47
 - event_, 49
 - font_, 49
 - GameOver, 47
 - pollEv, 48
 - render, 48
 - running, 48
 - state_, 49
 - text_, 49
 - update, 48
 - window_, 49
- gameTitle_
 - mainMenu, 56
- get_cord
 - Node, 64
- get_next
 - Node, 64
- getAttackSpeed
 - Tower, 82
- getDamage
 - Projectile, 75
- getEnemies
 - Wave, 89
- getEnemy
 - Projectile, 75
- getHealth
 - Player, 67
- getHealthText
 - Player, 67
- getHitbox
 - Enemy, 29
 - Projectile, 76
- getHitpoints
 - Enemy, 29
- getLastFireTime
 - Tower, 82
- getLevel
 - Tower, 82
- getMoney
 - Player, 67
- getMoneyText
 - Player, 67
- getNextWindow
 - mainMenu, 55
- getNodes
 - Map, 59
- getPoisoned
 - Enemy, 29
- getProjectiles
 - basicTower, 14
 - BombTower, 18
 - IceTower, 53
 - PoisonTower, 73
 - Tower, 82
- getRange
 - Tower, 83
- getRangeCircle
 - Tower, 83
- getSpeed
 - Enemy, 30
- getSprite
 - Button, 22
 - Enemy, 30
 - GameCell, 45
 - Projectile, 76
 - Tower, 83
- getSteps
 - Enemy, 30
- getTarget
 - Tower, 83
- getText
 - Button, 22
 - Tower, 84
- getTiles
 - Map, 59
- getType
 - Button, 22
 - Enemy, 30
 - Tower, 84
- giveMoney
 - Player, 67
- has_payed_
 - Enemy, 35
- has_target_
 - Tower, 86
- hasTarget
 - Tower, 84
- heal
 - Player, 68
- health_
 - Player, 69
- hitbox_
 - Enemy, 35
 - Projectile, 77
- hitpoints_

- Enemy, 35
- hp_text_
 - Player, 69
- IceProjectile, 50
 - IceProjectile, 51
- IceTower, 51
 - ~IceTower, 52
 - fire, 53
 - getProjectiles, 53
 - IceTower, 52
 - projectiles_, 53
- intersects
 - Game, 39
- is_clicked_
 - Button, 25
- is_Finnish
 - Button, 25
- is_Start
 - Button, 25
- isalive
 - Enemy, 31
- isClicked
 - Button, 22
- isExit_
 - GameCell, 45
- isFinnish
 - Button, 22
- isFrozen
 - Enemy, 31
- isPassable_
 - GameCell, 45
- isRunning
 - mainMenu, 55
 - mapCreator, 61
- isSelected
 - Tower, 84
- isStart
 - Button, 23
- isStart_
 - GameCell, 46
- isValid
 - pathfinding.cpp, 120
 - pathfinding.hpp, 122
- kill
 - Enemy, 31
- last_fire_time_
 - Tower, 86
- level_
 - Tower, 86
- main
 - main.cpp, 113
- main.cpp
 - main, 113
- mainMenu, 54
 - ~mainMenu, 54
- buttons_, 56
- event_, 56
- exitButton_, 56
- exitButtonPressed_, 56
- exitButtonSelected_, 56
- font_, 56
- gameTitle_, 56
- getNextWindow, 55
- isRunning, 55
- mainMenu, 54
- mapButton_, 56
- mapButtonPressed_, 57
- mapButtonSelected_, 57
- next_window_, 57
- playButton_, 57
- playButtonPressed_, 57
- playButtonSelected_, 57
- pollEv, 55
- Render, 55
- Update, 55
- window_, 57
- mainMenuButton_
 - mapCreator, 62
- Map, 58
 - ~Map, 59
 - file_path_, 59
 - getNodes, 59
 - getTiles, 59
 - Map, 58
 - nodes_, 60
 - setUpCells, 59
 - tile_matrix_, 60
- map_
 - Game, 42
- mapButton_
 - mainMenu, 56
- mapButtonPressed_
 - mainMenu, 57
- mapButtonSelected_
 - mainMenu, 57
- mapCreator, 60
 - ~mapCreator, 61
 - buttons_, 62
 - checkButtons, 61
 - event_, 62
 - isRunning, 61
 - mainMenuButton_, 62
 - mapCreator, 61
 - pollEv, 61
 - Render, 62
 - saveButton_, 63
 - setFinnishButton_, 63
 - setStartButton_, 63
 - showInfo, 62
 - Update, 62
 - window_, 63
- money_
 - Player, 69

- money_text_
 - Player, 69
- move
 - Enemy, 31
 - Projectile, 76
- moveTower
 - Game, 40
- next_
 - Node, 65
- next_point_
 - Enemy, 35
- next_window_
 - mainMenu, 57
- Node, 63
 - coordinate_, 65
 - get_cord, 64
 - get_next, 64
 - next_, 65
 - Node, 64
 - set_cord, 64
 - set_next, 64
- nodes_
 - Game, 42
 - Map, 60
- notClicked_
 - Button, 25
- pathfinding.cpp
 - isValid, 120
 - pathSearch, 121
- pathfinding.hpp
 - isValid, 122
 - pathSearch, 122
- pathSearch
 - pathfinding.cpp, 121
 - pathfinding.hpp, 122
- paused_
 - Game, 42
- placing_new_tower
 - Game, 42
- playButton_
 - mainMenu, 57
- playButtonPressed_
 - mainMenu, 57
- playButtonSelected_
 - mainMenu, 57
- Player, 65
 - font_, 69
 - getHealth, 67
 - getHealthText, 67
 - getMoney, 67
 - getMoneyText, 67
 - giveMoney, 67
 - heal, 68
 - health_, 69
 - hp_text_, 69
 - money_, 69
 - money_text_, 69
 - Player, 66
 - takeDamage, 68
 - takeMoney, 68
- player_
 - Game, 43
- poison_timer_
 - Enemy, 35
- poisoned_
 - Enemy, 35
- PoisonProjectile, 70
 - PoisonProjectile, 71
- PoisonTower, 71
 - ~PoisonTower, 72
 - fire, 73
 - getProjectiles, 73
 - PoisonTower, 72
 - projectiles_, 73
- pollEv
 - Game, 40
 - GameOver, 48
 - mainMenu, 55
 - mapCreator, 61
- pos_
 - GameCell, 46
- price_
 - Tower, 87
- Projectile, 74
 - ~Projectile, 75
 - damage_, 77
 - enemy_, 77
 - getDamage, 75
 - getEnemy, 75
 - getHitbox, 76
 - getSprite, 76
 - hitbox_, 77
 - move, 76
 - Projectile, 75
 - speed_, 77
 - sprite_, 77
 - texture_, 77
 - type_, 77
- projectiles_
 - basicTower, 14
 - BombTower, 18
 - IceTower, 53
 - PoisonTower, 73
- range_
 - Tower, 87
- range_circle_
 - Tower, 87
- removeTarget
 - Tower, 85
- Render
 - mainMenu, 55
 - mapCreator, 62
- render
 - Game, 40
 - GameOver, 48

- resetSelected
 - Tower, 85
- running
 - Game, 40
 - GameOver, 48
- saveButton_
 - mapCreator, 63
- selected_
 - Tower, 87
- set_cord
 - Node, 64
- set_next
 - Node, 64
- setCheckpoint
 - Enemy, 32
- setClicked
 - Button, 23
- setFinnish
 - Button, 23
- setFinnishButton_
 - mapCreator, 63
- setHitpoints
 - Enemy, 32
- setLastFireTime
 - Tower, 85
- setNotClicked
 - Button, 23
- setNotFinnish
 - Button, 23
- setNotStart
 - Button, 24
- setSelected
 - Tower, 85
- setSprite
 - Button, 24
- setStart
 - Button, 24
- setStartButton_
 - mapCreator, 63
- setTarget
 - Tower, 85
- setUpCells
 - Map, 59
- setUpSprite
 - GameCell, 45
- shouldPay
 - Enemy, 32
- showErrorMessage
 - Game, 40
- showInfo
 - Game, 41
 - mapCreator, 62
- speed_
 - Enemy, 35
 - Projectile, 77
- sprite_
 - Button, 25
 - Enemy, 36
 - GameCell, 46
 - Projectile, 77
 - Tower, 87
- src/Basic_enemy.cpp, 91
- src/Basic_enemy.hpp, 91
- src/Basic_projectile.cpp, 92
- src/Basic_projectile.hpp, 93
- src/Basic_tower.cpp, 94
- src/Basic_tower.hpp, 94
- src/Bomb_projectile.cpp, 95
- src/Bomb_projectile.hpp, 96
- src/Bomb_tower.cpp, 97
- src/Bomb_tower.hpp, 97
- src/Boss_enemy.cpp, 98
- src/Boss_enemy.hpp, 99
- src/Button.cpp, 100
- src/Button.hpp, 101
- src/Enemy.cpp, 102
- src/Enemy.hpp, 103
- src/Game.cpp, 104
- src/Game.hpp, 104
- src/Game_over.cpp, 105
- src/Game_over.hpp, 106
- src/gameCell.cpp, 108
- src/gameCell.hpp, 108
- src/Ice_projectile.cpp, 110
- src/Ice_projectile.hpp, 110
- src/Ice_tower.cpp, 111
- src/Ice_tower.hpp, 112
- src/main.cpp, 113
- src/mainMenu.cpp, 114
- src/mainMenu.hpp, 114
- src/Map.cpp, 115
- src/Map.hpp, 116
- src/mapCreator.cpp, 117
- src/mapCreator.hpp, 117
- src/Node.cpp, 119
- src/Node.hpp, 119
- src/pathfinding.cpp, 120
- src/pathfinding.hpp, 121
- src/Player.cpp, 123
- src/Player.hpp, 123
- src/Poison_projectile.cpp, 125
- src/Poison_projectile.hpp, 125
- src/Poison_tower.cpp, 126
- src/Poison_tower.hpp, 127
- src/Projectile.cpp, 128
- src/Projectile.hpp, 129
- src/readme.md, 129
- src/Stealth_enemy.cpp, 129
- src/Stealth_enemy.hpp, 130
- src/Tower.cpp, 131
- src/Tower.hpp, 132
- src/Wave.cpp, 133
- src/Wave.hpp, 133
- state_
 - Game, 43
 - GameOver, 49

- StealthEnemy, 78
 - StealthEnemy, 79
- steps_taken_
 - Enemy, 36
- takeDamage
 - Enemy, 32
 - Player, 68
- takeMoney
 - Player, 68
- target_
 - Tower, 87
- text_
 - Button, 25
 - GameOver, 49
 - Tower, 87
- texture_
 - Button, 25
 - Enemy, 36
 - GameCell, 46
 - Projectile, 77
 - Tower, 87
- tile_matrix_
 - Map, 60
- tiles_
 - Game, 43
- Tower, 79
 - ~Tower, 81
 - attack_speed_, 86
 - fire, 81
 - font_, 86
 - getAttackSpeed, 82
 - getLastFireTime, 82
 - getLevel, 82
 - getProjectiles, 82
 - getRange, 83
 - getRangeCircle, 83
 - getSprite, 83
 - getTarget, 83
 - getText, 84
 - getType, 84
 - has_target_, 86
 - hasTarget, 84
 - isSelected, 84
 - last_fire_time_, 86
 - level_, 86
 - price_, 87
 - range_, 87
 - range_circle_, 87
 - removeTarget, 85
 - resetSelected, 85
 - selected_, 87
 - setLastFireTime, 85
 - setSelected, 85
 - setTarget, 85
 - sprite_, 87
 - target_, 87
 - text_, 87
 - texture_, 87
 - Tower, 81
 - type_, 88
 - upgrade, 86
- towers_
 - Game, 43
- type_
 - Button, 25
 - Enemy, 36
 - Projectile, 77
 - Tower, 88
- unFreeze
 - Enemy, 34
- Update
 - mainMenu, 55
 - mapCreator, 62
- update
 - Game, 41
 - GameOver, 48
- upgrade
 - Tower, 86
- validPlacement
 - Game, 41
- Wave, 88
 - enemies_, 89
 - getEnemies, 89
 - Wave, 88
- wave_number_
 - Game, 43
- window_
 - Game, 43
 - GameOver, 49
 - mainMenu, 57
 - mapCreator, 63
- x_goal_
 - Enemy, 36
- y_goal_
 - Enemy, 36